TIENDAFAST

Documentación del proyecto

Plataforma de delivery multitenant para farmacias y negocios locales

Esquema de negocio · Recursos mínimos · Despliegue en VPS

Estado técnico, pruebas y plan de trabajo

2 de noviembre de 2026

Documento generado desde los archivos .md del repositorio

Índice

1. **Resumen ejecutivo** — estado, recursos, planes, márgenes y bloqueantes.
2. **Anexo A** — Guía de migración a VPS, recursos y esquema de negocio.
3. **Anexo B** — Plan de trabajo, pruebas y bitácora del proyecto.
4. **Anexo C** — Documentación completa del sistema.
5. **Anexo D** — Flujo completo de la aplicación.
6. **Anexo E** — Brochure comercial.
7. **Anexo F** — README del repositorio.

Los anexos conservan el contenido de sus archivos originales. Para el despliegue real, usa como referencia principal el Anexo A y el checklist de producción. El estado técnico más reciente está en el Anexo B.

Resumen ejecutivo

TiendaFast es una plataforma de delivery y gestión de inventario multitenant: una sola instalación atiende a varias farmacias o negocios, cada uno con su URL pública `/slug`, su catálogo, sus clientes, sus tasas, su delivery y sus datos de pago.

Estado del proyecto (2026-11-02)

- Fase 0 y Fase 1 completadas hasta **F1-34**.
- Identidad de clientes por tienda resuelta: un mismo correo o teléfono puede comprar en tiendas distintas sin mezclar pedidos.
- Pruebas: **34/34** unitarias de backend, **83/83** de API, **41/41** de frontend, typecheck y build correctos, lint con 0 errores.
- Pendiente antes de operar inventario real: restaurar stock al cancelar pedidos (F2-25), aprobación/rechazo de pagos (F2-01–F2-05), seguridad/auditoría (F3), backups y despliegue (F4) y E2E multi-usuario (F5).
- No hay módulo automático de planes, suscripciones o facturación fiscal. Para los primeros clientes se cobra y controla manualmente.

Recursos mínimos para los primeros clientes

Recurso	Mínimo técnico	Recomendado
CPU	2 vCPU	2 vCPU
RAM	2 GB + 2 GB swap	**4 GB**
Disco	40 GB SSD	80 GB NVMe
Transferencia	1 TB/mes	2 TB/mes
Sistema operativo	Ubuntu 22.04/24.04 LTS	Ubuntu 24.04 LTS
Costo estimado	$7–12/mes	$12–24/mes

Con 2 GB la app opera, pero compilar el frontend en la misma VPS puede agotar la memoria. Con 4 GB se puede compilar, correr Node + PostgreSQL + Nginx y hacer backups sin problema. Para 6–20 tiendas se recomienda 4 vCPU / 8 GB / 120 GB NVMe.

Esquema de negocio

Plan	Precio	Incluye
Implementación	$90 único	Configuración, catálogo, BCV, capacitación y salida en vivo
Emprendedor	$9/mes	1 tienda, 300 productos, 1 administrador
Negocio	$19/mes	Productos ilimitados, combos, 3 administradores, reportes
Pro	$35/mes	Dominio propio, administradores ilimitados, capacitación 1:1, soporte prioritario
Anualidad	10 meses por 12	Implementación gratis y precio congelado
Comisión por pedido	0%	Sin porcentaje sobre ventas

Cobro en bolívares a tasa BCV del día: Pago Móvil, transferencia, efectivo, Zelle o USDT. Piloto: 60 días gratis, sin implementación y 50% de descuento por 3 meses al continuar.

Margen con la infraestructura recomendada

Clientes	Ingreso mensual	Infraestructura	Margen bruto aproximado
1	$9–35	$15–50	Break-even
3	$27–105	$15–50	$0–90
5	$45–175	$24–50	$20–150
10	$90–350	$24–50	$40–300

Los primeros clientes pagan la VPS y el dominio. El costo real por tienda adicional es marginal. La implementación de $90 cubre el onboarding y la capacitación.

Cómo vender los primeros 3–5 clientes

1. Cobra la implementación y el primer mes por transferencia o Pago Móvil.
2. Lleva el control de plan, fecha de pago y límites en una hoja de cálculo.
3. Aplica los límites por política (no por software) hasta implementar la Fase 2B.
4. Onboardea una tienda a la vez y espera una semana de estabilidad antes de la siguiente.
5. Pide retroalimentación semanal y usa las métricas para el caso de éxito.

Bloqueantes antes de escalar a 10+ clientes

- Planes, límites y facturación automática (Fase 2B).
- Restauración de stock al cancelar (F2-25).
- Aprobación y rechazo de pagos en el panel (F2-01–F2-05).
- Backups probados y monitoreo (F4).
- Facturación fiscal y términos legales.

ANEXO A

Guia de Migracion a VPS — Plataforma Delivery Multitenant

1. Recursos y Costos

La app es **multitenant**: una sola instalacion atiende a todas las tiendas. No se crea una VPS ni un despliegue por farmacia; cada tienda es un registro en la base de datos con su `slug`, usuarios, catalogo, tasas, delivery y datos bancarios.

Primeros 1-5 clientes (recomendado)

Recurso	Minimo tecnico	Recomendado
CPU	2 vCPU	2 vCPU
RAM	2 GB + swap	4 GB
Disco	40 GB SSD	80 GB NVMe
Bandwidth	1 TB/mes	2 TB/mes
SO	Ubuntu 22.04/24.04 LTS	Ubuntu 24.04 LTS
Costo estimado	$7-12/mes	$12-24/mes

Importante: con 2 GB la app opera, pero compilar el frontend en la misma VPS puede quedarse sin memoria. Compila en tu maquina/CI y sube `frontend/dist`, o agrega 2 GB de swap. Con 4 GB puedes compilar, correr Node + PostgreSQL + Nginx y hacer backups sin problema.

6-20 tiendas (hasta ~100 pedidos/dia)

Recurso	Recomendado
CPU	4 vCPU
RAM	8 GB
Disco	120-160 GB NVMe
Bandwidth	3 TB/mes
Costo estimado	$24-50/mes

20-50 tiendas

Recurso	Recomendado
CPU	8 vCPU
RAM	16 GB
Disco	200 GB NVMe
Base de datos	PostgreSQL gestionado (Neon/RDS)
Costo estimado	$80-150/mes

Proveedores Recomendados

Proveedor	Ventaja	Desde
Hetzner	Mejor relacion precio/calidad, EU	$4.50/mes (2 vCPU, 4 GB)
DigitalOcean	Facil de usar, buena documentacion	$24/mes (2 vCPU, 4 GB)
Vultr	Globales, buen rendimiento	$24/mes (2 vCPU, 4 GB)
Contabo	Mucho disco y RAM barato	$7/mes (4 vCPU, 8 GB)
Oracle Cloud	Gratis tier (limitado)	$0 (1-4 ARM)

Recomendacion: Hetzner CX22/CX32 o Contabo VPS S para empezar. Si el publico esta en Venezuela, prioriza un proveedor con buena latencia hacia Miami/US-East (DigitalOcean/Vultr) o usa Cloudflare delante.

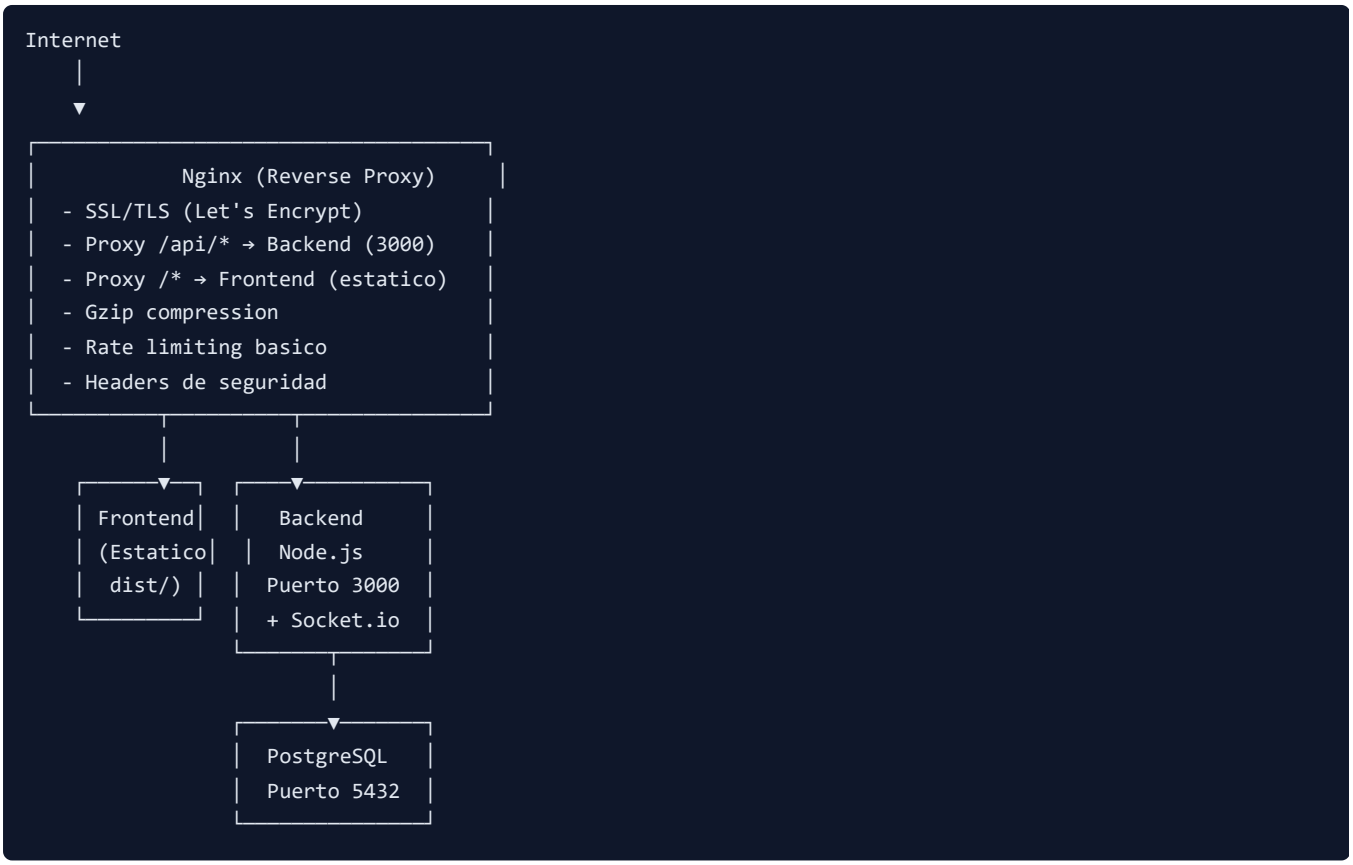
Costos mensuales tipicos (primeros clientes)

Concepto	Costo estimado
VPS 2 vCPU / 4 GB	$12-24
Dominio `.com`	$1-2
SSL Let's Encrypt	$0
Cloudinary (imagenes)	$0 (free tier)
Resend (correos OTP)	$0-20
Backups externos (S3/Backblaze)	$0-5
Total	**$15-50/mes**

Con un solo cliente de $19/mes ya cubres la infraestructura basica; con 3 clientes hay margen para soporte y publicidad.

---

2. Stack del Servidor



---

3. Pasos de Migracion

Paso 1: Preparar la VPS

```
# Conectar via SSH
ssh root@TU_IP_VPS

# Actualizar sistema
apt update && apt upgrade -y

# Instalar dependencias
apt install -y curl git ufw nginx certbot python3-certbot-nginx

# Configurar firewall
ufw allow OpenSSH
ufw allow 'Nginx Full'
ufw enable
```

Paso 2: Instalar Node.js y PostgreSQL

```
# Node.js 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs

# Verificar
node -v  # v20.x.x
npm -v   # 10.x.x

# PostgreSQL
apt install -y postgresql postgresql-contrib

# Iniciar y habilitar
systemctl start postgresql
systemctl enable postgresql
```

Paso 3: Configurar Base de Datos

```
# Entrar a PostgreSQL
su - postgres
psql

# Crear usuario y base de datos
CREATE USER delivery_user WITH PASSWORD 'TU_PASSWORD_SEGURO';
CREATE DATABASE delivery_db OWNER delivery_user;
GRANT ALL PRIVILEGES ON DATABASE delivery_db TO delivery_user;
\q

# Salir de postgres user
exit
```

Paso 4: Clonar el Proyecto

```
# Crear directorio
mkdir -p /var/www
cd /var/www

# Clonar repo
git clone https://github.com/JesusQuilleli/delivery-app.git
cd delivery-app
```

Paso 5: Configurar Backend

```
cd backend

# Instalar dependencias (reproducible)
npm ci

# Crear archivo .env
cat > .env << 'EOF'
PORT=3000
DATABASE_URL="postgresql://delivery_user:TU_PASSWORD_SEGURO@localhost:5432/delivery_db"
JWT_SECRET="genera_un_secreto_de_128_caracteres_aqui"
RESEND_API_KEY="tu_clave_resend"
CLOUDINARY_CLOUD_NAME="tu_cloud_name"
CLOUDINARY_API_KEY="tu_api_key"
CLOUDINARY_API_SECRET="tu_api_secret"
ALLOWED_ORIGINS="https://tudominio.com,https://www.tudominio.com"
EOF

# Generar cliente Prisma
npx prisma generate

# Aplicar migraciones a la DB (NO usar db push en produccion)
npx prisma migrate deploy

# Solo en desarrollo/demo: sembrar datos iniciales
# npm run db:seed

# Test: el backend debe funcionar
npm start
```

Paso 6: Configurar Frontend

```
cd /var/www/delivery-app/frontend

# Instalar dependencias (reproducible)
npm ci

# Crear archivo de produccion
cat > .env.production << 'EOF'
VITE_API_URL=https://tudominio.com/api
VITE_SOCKET_URL=https://tudominio.com
EOF

# Build de produccion
npm run build

# El resultado queda en frontend/dist/
```

Paso 7: Configurar Nginx

```
# Crear configuracion del sitio
cat > /etc/nginx/sites-available/delivery << 'EOF'
server {
    listen 80;
    server_name tudominio.com www.tudominio.com;

    # Frontend estatico
    location / {
        root /var/www/delivery-app/frontend/dist;
        index index.html;
        try_files $uri $uri/ /index.html;
    }

    # Backend API
    location /api/ {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }

    # Socket.io WebSocket
    location /socket.io/ {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 86400;
        proxy_send_timeout 86400;
    }

    # Compresion Gzip
    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml application/xml
text/javascript image/svg+xml;
    gzip_min_length 256;
    gzip_vary on;

    # Seguridad
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    # Cache de assets estaticos
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
        root /var/www/delivery-app/frontend/dist;
```

```
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
EOF

# Habilitar el sitio
ln -s /etc/nginx/sites-available/delivery /etc/nginx/sites-enabled/

# Eliminar configuracion default
rm /etc/nginx/sites-enabled/default

# Test de configuracion
nginx -t

# Recargar Nginx
systemctl reload nginx
```

Paso 8: SSL con Let's Encrypt

```
# Obtener certificado
certbot --nginx -d tudominio.com -d www.tudominio.com

# Auto-renovacion (ya viene configurada)
certbot renew --dry-run
```

Paso 9: Configurar PM2 (Proceso Manager)

```
# Instalar PM2
npm install -g pm2

# Iniciar backend con PM2
cd /var/www/delivery-app/backend
pm2 start server.js --name "delivery-api"

# Configurar auto-reinicio al reiniciar servidor
pm2 startup
pm2 save

# Comandos utiles
pm2 status              # Ver procesos
pm2 logs delivery-api   # Ver logs
pm2 restart delivery-api # Reiniciar
pm2 stop delivery-api    # Detener
```

Paso 10: Actualizar Frontend en el Repo

```
# En tu maquina local, actualizar .env.production
# frontend/.env.production:
# VITE_API_URL=https://tudominio.com/api
# VITE_SOCKET_URL=https://tudominio.com

# Push a GitHub
git add frontend/.env.production
git commit -m "chore: apuntar API a VPS"
git push
```

---

4. Comandos de Mantenimiento

Actualizar la app

```
cd /var/www/delivery-app
git pull origin main

# Backend
cd backend
npm ci
npx prisma generate
npx prisma migrate deploy
pm2 restart delivery-api

# Frontend (o compilar en CI y subir dist/)
cd ../frontend
npm ci
npm run build
```

Ver logs

```
pm2 logs delivery-api --lines 50
journalctl -u nginx --since "1 hour ago"
```

Backup de base de datos

```
# Backup manual
pg_dump -U delivery_user delivery_db > backup_$(date +%Y%m%d).sql

# Backup automatico (cron cada 24h)
crontab -e
# Agregar:
0 2 * * * pg_dump -U delivery_user delivery_db | gzip > /var/backups/delivery_$(date +\%Y\%m\%d).sql.gz
```

Monitoreo

```
# Estado del servidor
htop
df -h
free -m

# Estado de servicios
systemctl status nginx
pm2 status
systemctl status postgresql
```

---

5. Seguridad Adicional

Fail2Ban (proteccion SSH)

```
apt install -y fail2ban
systemctl enable fail2ban
```

Actualizaciones automaticas de seguridad

```
apt install -y unattended-upgrades
dpkg-reconfigure -plow unattended-upgrades
```

Limpiar backups viejos

```
# Agregar a cron
0 3 * * 0 find /var/backups -name "delivery_*" -mtime +30 -delete
```

---

6. Verificar que Todo Funciona

```
# 1. Backend responde
curl https://tudominio.com/api/
# Debe retornar: "API Delivery Frictionless esta corriendo..."

# 2. Frontend carga
curl -I https://tudominio.com/
# Debe retornar: HTTP/2 200

# 3. WebSocket conecta
# Abrir https://tudominio.com/admin-login en el navegador
# Login como admin → debe cargar el dashboard

# 4. Pedidos en tiempo real
# Abrir https://tudominio.com/farmacia-ayacucho en otro navegador
# Hacer un pedido → el dashboard debe sonar y mostrar el pedido
```

---

7. Configurar cada tienda (onboarding)

No se despliega una app por tienda. Todas las tiendas viven en la misma app y base de datos:

- Catalogo publico: `https://tudominio.com/<slug>` (ej. `https://tudominio.com/farmacia-ayacucho`).
- Panel del negocio: `https://tudominio.com/admin-login` (usuario y contraseña unicos del administrador).
- Superadmin: `https://tudominio.com/superadmin`.

Pasos para dar de alta una farmacia

1. Entrar como superadmin y crear la tienda con nombre, `slug` unico, moneda base, industria y credenciales del administrador.
2. Entrar como administrador y completar **Settings**:
   - Nombre y ubicacion GPS de la sucursal.
   - Moneda base (USD/VES/COP) y tasas (BCV o manuales).
   - Delivery: tarifa plana y zonas por rango de distancia.
   - Datos de pago: banco, codigo, telefono, cedula/RIF, titular y nota.
3. Cargar categorias y catalogo (productos, precios, stock, imagenes en Cloudinary).
4. Crear motorizados en **Drivers**.
5. Probar el flujo completo con un correo de prueba: agendar -> confirmar direccion -> pedido -> dashboard en tiempo real.
6. Entregar al dueno: URL publica, URL del panel, credenciales y guia rapida de 1 pagina.

Dominios

- **Recomendado para empezar:** un solo dominio con rutas por `slug` (lo que ya soporta la app).
- Subdominios por tienda (`farmacia.tudominio.com`) requieren DNS wildcard, Nginx con `server_name *.tudominio.com` y que el backend resuelva el `slug` desde el `Host`. **No esta implementado todavia**; no lo prometas a clientes sin desarrollarlo.
- Dominio propio (plan Pro) requiere el mismo trabajo de resolucion por host o un proxy inverso por tienda.

Aislamiento y limitaciones actuales

- Todos los datos de catalogo, pedidos, tasas, delivery y banco estan filtrados por `store_id`.
- **Identidad de clientes por tienda (resuelto en F1-29–F1-34):** un mismo correo y telefono pueden registrarse en tiendas distintas. El único es compuesto por `store_id`; dentro de una misma tienda el telefono no puede repetirse. Migracion `20261103022000_user_identity_per_store`.
- El usuario de administracion (`username`) sigue siendo unico global; el telefono del admin ya no impide crear otra tienda.
- No hay planes/suscripciones automaticas ni limites aplicados por el sistema (Fase 2B). Para los primeros clientes, cobra y controla limites manualmente.
- No hay facturacion fiscal venezolana; confirma con un contador antes de prometerla.

---

8. Pruebas con multiples usuarios

Preparacion

- Usa un entorno de staging con una copia de la base de datos (rama de Neon o `pg_dump` / `pg_restore`).
- Crea al menos: 2 tiendas, 3 clientes por tienda, 1 administrador por tienda, 1 superadmin y 2 motorizados.
- Prueba en 2 navegadores distintos + 2 celulares reales (no solo incognito).

Matriz minima

Caso	Resultado esperado
Dos clientes compran la ultima unidad a la vez	Solo uno recibe 200; el otro recibe 400 de stock
Cliente A intenta ver pedido de cliente B	403
Admin A consulta pedidos/clientes de tienda B	403
Mismo correo y telefono compra en tienda A y B	Permitido; cada compra queda asociada a su tienda y sus pedidos no se mezclan
Pedido en efectivo	Aparece en dashboard en < 2 s
Pedido por Pago Movil con referencia invalida	400 y mensaje claro
Cliente cancela `AWAITING_PAYMENT`	Estado `CANCELLED`
Tienda desactivada	Catalogo y checkout bloqueados
Zona de delivery coincide	Cobra la tarifa de la zona, no la plana
Reinicio de la VPS con pedidos activos	La app vuelve y los pedidos siguen en la DB

Carga

- Usa k6 o Artillery contra staging, no contra produccion.
- Escenario inicial: 50 usuarios virtuales navegando 5 min + 10 pedidos/minuto.
- Criterios: 0 errores 5xx, p95 de API < 500 ms, stock sin negativos, sin pedidos duplicados.
- Repite con 100 y 200 usuarios para conocer el limite antes de la temporada alta.

Cierre

- Cada bug encontrado se registra en `PLAN_DE_TRABAJO.md` y se prioriza.
- No se onboardea un cliente nuevo hasta pasar la matriz completa en staging.

---

9. Checklist de produccion

- [ ] `NODE_ENV=production`, `JWT_SECRET` de 128 caracteres, `.env` fuera de git.
- [ ] `npx prisma migrate deploy` (nunca `db push`) y backup antes de cada migracion.
- [ ] HTTPS con renovacion automatica, HSTS, CORS limitado al dominio real.
- [ ] Rate limiting activo en auth, pedidos y admin.
- [ ] PM2 con reinicio automatico y rotacion de logs (`pm2 install pm2-logrotate`).
- [ ] 2 GB de swap si la VPS tiene 2 GB de RAM; build del frontend fuera del servidor.
- [ ] Backup diario de PostgreSQL + copia externa (S3/Backblaze) + prueba de restauracion mensual.
- [ ] Monitoreo de uptime (UptimeRobot/BetterStack) y alertas por correo/Telegram.
- [ ] Cloudinary configurado con limites de tamano y formatos.
- [ ] Dominio de Resend verificado (SPF/DKIM) para que los OTP no caigan en spam.
- [ ] Cloudflare delante del dominio (opcional pero recomendado).
- [ ] Terminos, privacidad y tratamiento de datos redactados.
- [ ] Matriz de pruebas multi-usuario aprobada.
- [ ] Stock restaurado al cancelar pedidos (F2-25) antes de operar inventario real.

---

10. Esquema de negocio para vender

Planes (alineados con `PLAN_DE_TRABAJO.md`)

Plan	Precio	Incluye
Implementacion	$90 unico	Configuracion, catalogo, BCV, capacitacion y salida en vivo
Emprendedor	$9/mes	1 tienda, 300 productos, 1 administrador
Negocio	$19/mes	Productos ilimitados, combos, 3 administradores, reportes
Pro	$35/mes	Dominio propio, administradores ilimitados, capacitacion 1:1, soporte prioritario
Anualidad	10 meses por 12	Implementacion gratis y precio congelado
Comision por pedido	0%	Sin porcentaje sobre ventas

- Cobro en bolivares a tasa BCV del dia: Pago Movil, transferencia, efectivo, Zelle o USDT.
- Piloto: 60 dias gratis, sin implementacion, con 50% de descuento por 3 meses al continuar.
- Revision de precios trimestral con aviso de 30 dias; sin contratos largos.

Margen con la infraestructura recomendada

Clientes	Ingreso mensual	Infraestructura	Margen bruto aproximado
1	$9-35	$15-50	Break-even
3	$27-105	$15-50	$0-90
5	$45-175	$24-50	$20-150
10	$90-350	$24-50	$40-300

Los primeros clientes pagan la VPS y el dominio. El costo real por tienda adicional es marginal (almacenamiento y correos). La implementacion de $90 cubre el tiempo de onboarding y capacitacion.

Como vender los primeros 3-5 clientes sin modulo de suscripciones

1. Cobra la implementacion y el primer mes por transferencia/Pago Movil.
2. Lleva el control de plan, fecha de pago y limites en una hoja de calculo.
3. Aplica los limites por politica (no por software) hasta implementar Fase 2B.
4. Onboardea una tienda a la vez y espera 1 semana de estabilidad antes de la siguiente.
5. Pide retroalimentacion semanal y usa las metricas para el caso de exito.

Bloqueantes antes de escalar a 10+ clientes

- Planes, limites y facturacion automatica (Fase 2B).
- Restauracion de stock al cancelar (F2-25).
- Aprobacion/rechazo de pagos en el panel (F2-01–F2-05).
- Backups probados y monitoreo (F4).
- Facturacion fiscal y terminos legales.

Guia generada para Plataforma Delivery Multitenant v2.0

ANEXO B

Plan de trabajo — TiendaFast

> Documento vivo. Actualizar después de cada tarea. Si se pierde la sesión: leer este archivo, revisar `git status` y `git log`, y continuar desde la primera casilla sin marcar.

- **Última actualización:** 2026-11-02
- **Rama actual:** `feature/security-hardening`
- **Rama principal:** `main`
- **Estado general:** F1-34 completada; siguiente F2-01 y F2-25
- **Zona del piloto:** Puerto Ayacucho, Amazonas
- **Segmento:** farmacias independientes y cadenas pequeñas

---

1. Cómo usar este documento

1. Antes de empezar una tarea, marcarla como `En curso` en el registro de avance.
2. Al terminarla, cambiar `- [ ]` a `- [x]`, agregar una línea al registro y hacer commit.
3. No borrar entradas anteriores del registro; solo agregar.
4. Si una tarea queda bloqueada, marcarla como `Bloqueado` y explicar el motivo.
5. Cada commit debe incluir la fase o el identificador de la tarea: `fix(cliente): restaurar sesión desde cookie httpOnly (F1-01)`.
6. Nunca subir `.env`, secretos, contraseñas ni claves API al repositorio.
7. Al finalizar cada fase, ejecutar pruebas y actualizar "Estado de pruebas".

Cómo retomar después de perder la sesión

```
git status
git log --oneline -15
git branch --show-current
```

Luego buscar en este archivo la primera tarea sin marcar y continuar. Si hay cambios sin commitear, revisar el diff antes de seguir:

```
git diff
```

---

2. Contexto del proyecto

Plataforma de delivery y gestión de inventario multitenant para negocios venezolanos. El piloto se enfoca en farmacias de Puerto Ayacucho.

- **Frontend:** React 19 + TypeScript + Vite + Tailwind.
- **Backend:** Node.js + Express + Prisma + PostgreSQL + Socket.io.
- **Multimedia:** Cloudinary.
- **Correo:** Resend.

- **Autenticación:** JWT en cookie httpOnly, OTP por correo para clientes y usuario/contraseña para administradores.
- **Producción actual documentada:** `https://test.shop-mg.com` con API en Render.
- **Producción objetivo:** VPS propio con Nginx, frontend y API en el mismo dominio.

Estado real del repositorio

- El núcleo funcional existe: catálogo, carrito, checkout, OTP, pedidos, dashboard, inventario, combos, motorizados, historial y SuperAdmin.
- La rama `feature/security-hardening` tiene mejoras sin commitear de F1-27/F1-28 (bloqueo de tiendas inactivas y admin como cliente).
- Las migraciones Prisma ya tienen baseline; precios y totales usan `Decimal(12,2)` y las tasas `Decimal(14,6)`.
- No hay CI, healthchecks, backups automatizados ni pruebas E2E.
- La documentación y los PDFs actuales prometen funciones que aún no están implementadas.
- No existe un módulo de planes, suscripciones, límites ni cobros.

---

3. Decisiones cerradas

Tema	Decisión
Modelo comercial	Suscripción mensual + implementación única
Precios	Emprendedor $9 / Negocio $19 / Pro $35
Implementación	$90; $60 para los primeros 5 clientes; gratis con anualidad
Anualidad	10 meses por 12; precio congelado
Comisión por pedido	0%
Prueba	14 días
Piloto	60 días gratis, implementación incluida
Después del piloto	50% de descuento por 3 meses
Indexación	Precios en USD; cobro en Bs al BCV del día; revisión trimestral
Producción	VPS con Nginx, mismo dominio para frontend y API
Clientes	Una cuenta por tienda; mismo correo puede existir en varias tiendas
Ventas y soporte	El fundador
Facturación fiscal	Se define con contador antes del primer cobro
Recetas y lotes	Se validan con farmacias antes de construir

---

4. Esquema de negocio

4.1 Cliente ideal

Farmacia independiente o cadena pequeña de Puerto Ayacucho, con 1–3 sucursales, 500–3.000 productos, pedidos por WhatsApp y delivery propio o tercerizado. Dueño o gerente accesible.

4.2 Precios

Plan	Precio	Límites
Implementación	$90 único	Configuración, catálogo, BCV, capacitación y salida en vivo
Emprendedor	$9/mes	1 tienda, 300 productos, 1 administrador
Negocio	$19/mes	Productos ilimitados, combos, 3 administradores, reportes
Pro	$35/mes	Dominio propio, administradores ilimitados, capacitación 1:1, soporte prioritario
Anualidad	10 meses por 12	Implementación gratis y precio congelado
Comisión por pedido	0%	Sin porcentaje sobre ventas

Cobro en bolívares a tasa BCV del día. Métodos: Pago Móvil, transferencia, efectivo, Zelle o USDT. Revisión trimestral con aviso de 30 días. Sin contratos largos; cancelación con 30 días de aviso.

4.3 Piloto en Puerto Ayacucho

- 60 días gratis, sin implementación.
- Capacitación y soporte directo del fundador.
- Tres sesiones de retroalimentación.
- Permiso para usar métricas reales de forma anonimizada.
- Continuidad con 50% de descuento por 3 meses.

Meta: lista de 15–20 farmacias, 8 contactos efectivos, 5 demos y 3 acuerdos firmados.

Canales: visitas presenciales, referidos de distribuidores, visitadores médicos y motorizados, WhatsApp e Instagram local, grupos de dueños de farmacias y tienda demo.

Criterios de selección: dueño o gerente con decisión, pedidos por WhatsApp, delivery, catálogo exportable y disposición a dar retroalimentación semanal.

4.4 Unit economics ilustrativos

Supuesto: 30 clientes (15 Emprendedor, 10 Negocio, 5 Pro) y 2 implementaciones al mes.

Concepto	Mensual
MRR	$500
Implementaciones	$180
Infraestructura, correo y respaldo	$50–80
Soporte (20 h)	$100
Comisiones de cobro (~2%)	$14
Margen bruto estimado	$480–500
Punto de equilibrio	10–12 clientes

Escenarios de planificación, no proyecciones garantizadas. Puerto Ayacucho sirve para validar, no para escalar; el crecimiento posterior requiere otra ciudad.

4.5 Fiscalidad

- Consultar contador en la semana 1 sobre IVA, IGTF, facturación digital y tratamiento de servicios en USD/BCV.
- El piloto es gratuito: no se factura hasta tener la estructura definida.
- No lanzar el cobro público sin resolución fiscal.

4.6 Requisitos específicos de farmacia

El esquema actual no tiene recetas, lotes ni vencimientos.

Requisito	Situación	Decisión
Receta médica	No implementado	Validar con 5 farmacias
Lotes y vencimientos	No implementado	Validar con 5 farmacias
Medicamentos controlados	No implementado	Excluir del piloto
Factura fiscal con IVA	No implementado	Definir con contador
Alertas de vencimiento	No implementado	Evaluar Fase 2C

Si 3 o más farmacias exigen recetas o lotes, agregar la Fase 2C (3–5 días).

---

5. Problemas conocidos

ID	Problema	Prioridad
C-01	La sesión no se restaura al abrir nueva pestaña o reiniciar navegador	Crítica
C-02	Checkout usa `current_store_id` de `localStorage` en vez del slug	Crítica
C-03	El carrito es global y no está aislado por tienda	Crítica
C-04	No se descuenta ni reserva stock; combos no validan componentes	Crítica
C-05	`User.email` y `User.phone` son únicos globalmente	Crítica
S-01	Salas Socket.io sin autenticación	Crítica
S-02	OTP en texto plano y sin límite de intentos	Alta
S-03	Cookies cross-site sin CSRF ni `trust proxy`	Alta
S-04	Datos de Pago Móvil hardcodeados	Alta
A-01	No existe flujo real de aprobación de pago	Alta
A-02	Configuración de tienda incompleta	Alta
A-03	Referencias cruzadas entre tiendas en productos/combos/categorías	Alta
A-04	`markOrderViewed` sin validación de tienda	Media
A-05	No existen planes, suscripciones, límites ni cobros	Crítica
P-01	Multi-moneda incorrecta para base VES/COP	Alta
P-02	Sin CI de API, healthchecks ni backups automatizados; migraciones ya iniciadas	Crítica
P-03	Tiendas inactivas siguen operando	Alta
P-04	Sin pruebas E2E ni pruebas de carga	Media
P-05	Sin zonas ni tarifas de delivery configurables	Alta
P-06	Lint con 85 advertencias: `any`, efectos con setState y exports de UI	Media
P-07	Bundle principal de ~1 MB sin code splitting	Media
P-08	Las pruebas de API requieren esperar a que el servidor esté listo	Baja
P-09	Resend rechaza correos `example.com`; las pruebas usan el código de la DB	Baja

6. Fases y tareas

Fase 0 — Orden y línea base (1 día)

- [x] Crear este documento.
- [x] Revisar todos los cambios sin commitear de `feature/security-hardening`.
- [x] Commitear la línea base (`c7c0377`).
- [x] Crear `backend/.env.example` sin secretos.
- [x] Crear `frontend/.env.example`.
- [x] Agregar script `typecheck` al frontend.
- [x] Separar scripts de pruebas unitarias y de API en backend.
- [x] Crear CI mínimo (frontend + backend unitario).
- [x] Ejecutar lint, typecheck, tests unitarios y build.
- [x] Ejecutar pruebas de API con PostgreSQL.
- [x] Registrar resultados y commitear Fase 0.

Resultado:

- Backend unitario: 23/23.
- Backend API: 63/63.
- Frontend: 32/32.
- Lint: 0 errores, 89 advertencias.
- Typecheck: correcto.
- Build: correcto; advertencia de bundle de ~1 MB.
- Se corrigieron errores de typecheck en `Footer`, `Landing`, `RestaurantTemplate` y `SupermarketTemplate`.
- Se corrigieron errores de lint en `AuthContext`, `SuperAdmin`, `Checkout`, `sidebar` y `eslint.config.js`.
- Se ajustó la limpieza de `inventory.api.test.js` para respetar la FK de `OrderItem`.
- Las pruebas de API requieren esperar a que el servidor escuche antes de ejecutarse.
- No se ejecutó `db:seed` para no borrar los 501 productos y 15 pedidos existentes.
- El CI de API quedará para la Fase 5; por ahora corre frontend y pruebas unitarias.

Criterio de terminado: rama estable, CI verde, entorno reproducible y secretos fuera del repositorio.

Fase 1 — Bugs críticos del cliente (5 días)

Sesión y contexto de tienda

- [x] F1-01 Restaurar sesión con `/auth/refresh` al iniciar la app.
- [x] F1-02 Manejar expiración real (401) sin cerrar sesión por errores de red.
- [x] F1-03 Derivar tienda, moneda y configuración desde el `slug`.
- [x] F1-04 Eliminar la dependencia de `current_store_id` en `localStorage`.
- [x] F1-05 Verificar checkout directo sin visitar el catálogo.
- [x] F1-06 Corregir redirección de `/mis-pedidos` para conservar el slug.

Carrito y stock

- [x] F1-07 Aislar el carrito por tienda.
- [x] F1-08 Limpiar o confirmar carrito al cambiar de tienda.
- [x] F1-09 Validar disponibilidad y stock antes de agregar o confirmar.

- [x] F1-10 Impedir cantidades superiores al stock.
- [x] F1-11 Descontar o reservar stock al crear el pedido en una transacción.
- [x] F1-12 Validar y descontar stock de componentes de combos.
- [x] F1-13 Rechazar pedidos con stock insuficiente.

Dinero y monedas

- [x] F1-14 Migrar precios y totales de `Float` a `Decimal` o centavos.
- [x] F1-15 Agregar al pedido: moneda, tasas, subtotal, delivery y total.
- [x] F1-16 Guardar snapshot de tasas para el historial.
- [x] F1-17 Corregir conversiones cuando la tienda base es VES o COP.
- [x] F1-18 Corregir formato de precios en COP.
- [x] F1-19 Integrar BCV desde backend con caché y endpoint de sincronización.
- [x] F1-20 Permitir override manual de tasas.
- [x] F1-21 Configurar zonas y tarifas de delivery.

Pagos y checkout

- [x] F1-22 Agregar datos bancarios por tienda al esquema.
- [x] F1-23 Eliminar datos bancarios hardcodeados de `Checkout.tsx`.
- [x] F1-24 Mostrar datos bancarios reales de la tienda.
- [x] F1-25 Validar referencia de Pago Móvil.
- [x] F1-26 Permitir cancelar pedidos `AWAITING_PAYMENT`.
- [x] F1-27 Bloquear compras en tiendas inactivas.
- [x] F1-28 Evitar que un administrador compre como cliente.

Identidad de clientes por tienda

- [x] F1-29 Quitar `@unique` global de `User.email` y `User.phone`.
- [x] F1-30 Agregar `@@unique([store_id, email])` y `@@unique([store_id, phone])`.
- [x] F1-31 Actualizar `checkEmail` y `verifyOtp` para buscar por `store_id + email`.
- [x] F1-32 Actualizar consultas que usan `findUnique({ where: { email } })`.
- [x] F1-33 Crear migración Prisma con respaldo previo.
- [x] F1-34 Probar que un mismo correo compra en dos tiendas sin mezclar datos.

Criterio de terminado: compra segura, sin sobreventa, precios correctos y cuentas aisladas por tienda.

Fase 2 — Bugs críticos del administrador (4 días)

Pagos y pedidos

- [ ] F2-01 Crear pestaña o filtro real para `AWAITING_PAYMENT`.
- [ ] F2-02 Aprobar pago: `AWAITING_PAYMENT` → `PENDING`.
- [ ] F2-03 Rechazar pago: `AWAITING_PAYMENT` → `CANCELLED`.
- [ ] F2-04 Soportar aprobación/rechazo por lote.
- [ ] F2-05 Corregir acciones que aceptan pedidos sin pago aprobado.
- [ ] F2-06 Agregar paginación y filtros server-side en pedidos.
- [ ] F2-07 Validar pertenencia de tienda en `markOrderViewed`.

Configuración de tienda

- [ ] F2-08 Permitir que SuperAdmin edite configuración.

- [ ] F2-09 Editar industria, tema, nombre, coordenadas y WhatsApp.
- [ ] F2-10 Editar datos bancarios y métodos de pago.
- [ ] F2-11 Implementar cambio de contraseña de administrador.
- [ ] F2-12 Implementar cierre de sesiones.
- [ ] F2-13 Quitar o completar secciones "en desarrollo" de `Settings.tsx`.

Inventario y combos

- [ ] F2-14 Validar que `category_id` pertenezca a la tienda.
- [ ] F2-15 Validar que los componentes de un combo pertenezcan a la tienda.
- [ ] F2-16 Calcular stock de combo como mínimo de `stock_componente / cantidad`.
- [ ] F2-17 Mostrar stock de combo en catálogo y checkout.
- [ ] F2-18 Evitar stock negativo.
- [ ] F2-19 Manejar eliminación de productos usados en combos.
- [ ] F2-20 Agregar alertas de bajo stock.
- [ ] F2-25 Restaurar stock al cancelar o eliminar pedidos (cliente, admin y lote).

Historial y analytics

- [ ] F2-21 Corregir filtros de fecha según zona horaria.
- [ ] F2-22 Calcular ventas netas y totales coherentes.
- [ ] F2-23 Corregir analytics con filtros aplicados.
- [ ] F2-24 Verificar exportación e impresión de reportes.

Tiendas inactivas

- [ ] F2-25 Bloquear catálogo y checkout de tiendas inactivas.
- [ ] F2-26 Bloquear recepción de pedidos en tiendas inactivas.
- [ ] F2-27 Definir acceso del admin de tienda inactiva.
- [ ] F2-28 Auditar activaciones y desactivaciones.

Criterio de terminado: operación completa sin estados inconsistentes ni accesos cruzados.

Fase 2B — Planes, suscripciones y onboarding SaaS (3 días)

- [ ] F2B-01 Agregar a `Store`: plan, ciclo, precio, estado, fin de prueba y próximo pago.
- [ ] F2B-02 Crear historial de pagos: monto, método, referencia, período y fecha.
- [ ] F2B-03 Crear panel SuperAdmin de clientes, planes y estados.
- [ ] F2B-04 Implementar marcar pagado, extender, suspender y cambiar plan.
- [ ] F2B-05 Aplicar límites por plan: productos y administradores.
- [ ] F2B-06 Vencimiento automático de prueba y suspensión por impago.
- [ ] F2B-07 Crear asistente de alta: tienda, admin, categorías y catálogo.
- [ ] F2B-08 Crear plantilla de recibo y recordatorios de pago.
- [ ] F2B-09 Crear panel de MRR y clientes activos.
- [ ] F2B-10 Registrar auditoría de acciones comerciales.

Criterio de terminado: se puede cobrar, suspender y administrar clientes sin planillas externas.

Fase 2C — Requisitos de farmacia (condicional, 0–5 días)

Solo si el discovery con 5 farmacias lo confirma.

- [ ] F2C-01 Validar necesidad de receta médica y lotes/vencimiento.
- [ ] F2C-02 Implementar carga y revisión de recetas.
- [ ] F2C-03 Implementar lotes y fechas de vencimiento.
- [ ] F2C-04 Implementar alertas de vencimiento.
- [ ] F2C-05 Bloquear venta de productos vencidos.
- [ ] F2C-06 Documentar exclusión de medicamentos controlados.

Fase 3 — Seguridad (4 días)

Socket.io

- [ ] F3-01 Autenticar el handshake con la cookie httpOnly.
- [ ] F3-02 Autorizar `join_store` según el token.
- [ ] F3-03 Autorizar `join_client` según el usuario.
- [ ] F3-04 Validar y limitar payloads.
- [ ] F3-05 Activar `withCredentials` en el cliente.
- [ ] F3-06 Probar aislamiento de salas.

Cookies y CSRF

- [ ] F3-07 Configurar `app.set('trust proxy', 1)`.
- [ ] F3-08 Migrar a mismo origen con `SameSite=Lax`.
- [ ] F3-09 Implementar CSRF si se mantiene cross-site.
- [ ] F3-10 Revisar expiración y rotación de JWT.
- [ ] F3-11 Implementar revocación de sesión.
- [ ] F3-12 Mantener `Secure` en producción.

OTP y autenticación

- [ ] F3-13 Guardar OTP con hash.
- [ ] F3-14 Limitar intentos por código.
- [ ] F3-15 Bloquear temporalmente tras fallos.
- [ ] F3-16 No devolver nombre ni teléfono en `checkEmail`.
- [ ] F3-17 Usar respuestas genéricas.
- [ ] F3-18 No imprimir OTP en logs.
- [ ] F3-19 Aplicar política de contraseñas.
- [ ] F3-20 Eliminar contraseñas demo de producción.

Validación y uploads

- [ ] F3-21 Schemas Zod para settings, clientes, motorizados y SuperAdmin.
- [ ] F3-22 Validar IDs, slugs y campos permitidos.
- [ ] F3-23 Evitar asignación masiva.
- [ ] F3-24 Limitar tamaño y tipo real de archivos.
- [ ] F3-25 Bloquear SVG.
- [ ] F3-26 Revisar permisos de Cloudinary.

Infraestructura

- [ ] F3-27 Helmet con CSP y HSTS.
- [ ] F3-28 Manejador centralizado de errores.

- [ ] F3-29 Request ID y auditoría.
- [ ] F3-30 `npm audit` y correcciones.
- [ ] F3-31 Dependabot.
- [ ] F3-32 Matriz de pruebas de autorización/IDOR.
- [ ] F3-33 Pruebas de sockets, OTP y CSRF.

Fase 4 — Producción en VPS (3 días)

- [ ] F4-01 Crear migraciones Prisma reales.
- [ ] F4-02 Dejar de usar `db push` en producción.
- [ ] F4-03 Preparar VPS: Node 20, PostgreSQL, Nginx, PM2, UFW, Fail2Ban.
- [ ] F4-04 Configurar frontend y API en el mismo dominio.
- [ ] F4-05 SSL con Let's Encrypt y renovación automática.
- [ ] F4-06 Configurar `NODE_ENV=production` y `ALLOWED_ORIGINS`.
- [ ] F4-07 Backups diarios de PostgreSQL.
- [ ] F4-08 Copiar backups fuera del VPS.
- [ ] F4-09 Probar restauración.
- [ ] F4-10 Crear `/health` y `/ready`.
- [ ] F4-11 Logs con rotación.
- [ ] F4-12 Monitoreo externo y alertas.
- [ ] F4-13 Pipeline de despliegue y rollback.
- [ ] F4-14 Índices de base de datos.
- [ ] F4-15 Runbook de incidentes.
- [ ] F4-16 Términos, privacidad y retención de datos.

Fase 5 — QA y aceptación (2 días)

- [ ] F5-01 Configurar Playwright.
- [ ] F5-02 E2E: registro/OTP en modo test.
- [ ] F5-03 E2E: pedido en efectivo.
- [ ] F5-04 E2E: pedido por transferencia y aprobación.
- [ ] F5-05 E2E: aceptar, despachar y entregar.
- [ ] F5-06 E2E: cancelar y calificar.
- [ ] F5-07 E2E: inventario y combos.
- [ ] F5-08 E2E: cambio de tienda y restauración de sesión.
- [ ] F5-09 E2E: permisos y aislamiento.
- [ ] F5-10 Pruebas de carga con k6.
- [ ] F5-11 Lighthouse y accesibilidad.
- [ ] F5-12 Revisión móvil.
- [ ] F5-13 Checklist de aceptación y go/no-go.

Fase 6 — PDFs y documentación (2 días)

PDF A — Propuesta Comercial

- [ ] F6-01 Crear `docs/comercial/propuesta-comercial.html`.

- [ ] F6-02 Portada y promesa de valor.
- [ ] F6-03 Problema del comercio venezolano.
- [ ] F6-04 Solución y diferenciadores.
- [ ] F6-05 Planes y precios.
- [ ] F6-06 Pago e indexación.
- [ ] F6-07 Retorno de inversión.
- [ ] F6-08 Implementación y soporte.
- [ ] F6-09 CTA con WhatsApp/QR.

PDF B — Beneficios

- [ ] F6-10 Crear `docs/comercial/beneficios-suscripcion.html`.
- [ ] F6-11 Cinco problemas actuales.
- [ ] F6-12 Ocho beneficios.
- [ ] F6-13 Funciones por plan.
- [ ] F6-14 Cómo empezar.
- [ ] F6-15 Preguntas frecuentes.
- [ ] F6-16 CTA y contacto.

Producción y cierre

- [ ] F6-17 Crear `scripts/generate-pdfs.ps1`.
- [ ] F6-18 Generar PDFs con Edge headless.
- [ ] F6-19 Revisar ortografía, precios y claims.
- [ ] F6-20 Verificar que no se prometan funciones no implementadas.
- [ ] F6-21 Actualizar `README.md` y documentación técnica.
- [ ] F6-22 Archivar PDFs antiguos.

---

7. Puertas de lanzamiento

- 3 farmacias usando el sistema durante 60 días.
- Pedidos reales sin errores críticos.
- Stock y vencimientos bajo control, si se implementan.
- Facturación fiscal definida antes de cobrar.
- Términos, privacidad y contrato listos.
- Backups restaurados y monitoreo activo.
- Aislamiento entre tiendas y seguridad verificados.
- PDFs alineados con funciones reales.

---

8. Comandos de referencia

Frontend

```
cd frontend
npm install
npm run dev
npm run lint
npm run typecheck
npm run build
npm test
```

Backend

```
cd backend
npm install
npx prisma generate
npx prisma migrate dev
npm run dev
npm run test:unit
npm run db:seed
npm run test:api
```

Pruebas de API

Requieren:

- Servidor backend corriendo en `http://localhost:3000`.
- PostgreSQL accesible.
- Datos demo sembrados.
- `JWT_SECRET` configurado.

Advertencia: `npm run db:seed` elimina pedidos, productos y categorías. No ejecutar en una base con datos reales.

Producción

```
cd /var/www/delivery-app
git pull origin main
cd backend
npm ci
npx prisma generate
npx prisma migrate deploy
pm2 restart delivery-api
cd ../frontend
npm ci
npm run build
```

---

9. Estado de pruebas

Fecha	Backend unitario	Backend API	Frontend	Lint	Typecheck	Build	Notas
2026-11-01	23/23	68/68	39/39	0 errores, 85 warnings	Correcto	Correcto	Fase 0 y F1-01–F1-13 completadas. Stock transaccional y combos. Bundle ~1 MB.
2026-11-02	33/33	77/77	41/41	0 errores, 85 warnings	Correcto	Correcto	F1-14–F1-26 y F1-27/F1-28. Decimal, tasas BCV, delivery por zonas, datos bancarios por tienda. Bundle ~1 MB.
2026-11-02	34/34	83/83	41/41	0 errores, 85 warnings	Correcto	Correcto	F1-29–F1-34. Identidad de clientes por tienda, mismo correo/teléfono en dos tiendas, compra aislada y timeout de transacción a 20 s. Bundle ~1 MB.

---

10. Registro de avance

Fecha	Fase / ID	Estado	Notas
2026-11-01	Plan creado	Hecho	Decisiones comerciales y técnicas cerradas
2026-11-01	Fase 0	Hecho	Línea base `c7c0377`, `.env.example`, scripts y CI
2026-11-01	Fase 0 pruebas	Hecho	Backend 86/86, frontend 32/32, lint sin errores, build OK
2026-11-01	F1-01	Hecho	Refresh al montar, `isInitializing` y espera en `ProtectedRoute`; no cierra sesión por errores de red
2026-11-01	F1-02	Hecho	Solo 401 limpia sesión; red y 500 conservan usuario; 2 pruebas nuevas
2026-11-01	F1-03	Hecho	Checkout usa `storeConfig.id`; moneda y tasas tipadas desde el slug
2026-11-01	F1-04	Hecho	Eliminado `current_store_id` de `localStorage` y de `Catalog`
2026-11-01	F1-05	Hecho	Checkout directo funciona con `storeConfig.id`; carrito persiste por tienda
2026-11-01	F1-06	Hecho	`/mis-pedidos` conserva el slug y espera la restauración de sesión
2026-11-01	F1-07	Hecho	`CartProvider` dentro del router; carrito separado por slug
2026-11-01	F1-08	Hecho	Al cambiar de tienda se carga su carrito; no se mezclan productos
2026-11-01	F1-09	Hecho	Frontend bloquea agotados; backend revalida antes de crear
2026-11-01	F1-10	Hecho	Carrito y botones respetan stock; no se supera el máximo

Fecha	Fase / ID	Estado	Notas
2026-11-01	F1-11	Hecho	Descuento condicional dentro de `$transaction`
2026-11-01	F1-12	Hecho	Combos descuentan componentes agregados por producto
2026-11-01	F1-13	Hecho	`StockError` devuelve 400 con detalle y hace rollback
2026-11-02	F1-14	Hecho	`Decimal(12,2)` para precios/totales y `Decimal(14,6)` para tasas; baseline de migraciones creado
2026-11-02	F1-15	Hecho	Pedido guarda moneda, subtotal, `delivery_fee` y total; `delivery_fee` inicia en 0
2026-11-02	F1-16	Hecho	Snapshot de `currency`, `usd_rate`, `ves_rate` y `cop_rate` en cada pedido
2026-11-02	F1-17	Hecho	Conversiones con pivote USD; funciona con base USD, VES o COP
2026-11-02	F1-18	Hecho	COP sin decimales y con separador de miles es-CO
2026-11-02	F1-19	Hecho	Servicio BCV con caché 10 min, `GET /api/rates/bcv` y `POST /stores/:slug/rates/sync-bcv`; fallback a tasa stale
2026-11-02	F1-20	Hecho	Tasas VES/COP editables manualmente en Settings y persistidas con el resto de la configuración
2026-11-02	F1-21	Hecho	`DeliveryZone` por tienda, tarifa plana, cotización pública y cálculo de `delivery_fee` en el pedido
2026-11-02	F1-22	Hecho	Campos `bank_*` y `payment_note` en `Store` con migración
2026-11-02	F1-23	Hecho	Checkout sin banco/teléfono/cédula hardcodeados
2026-11-02	F1-24	Hecho	Settings edita y el catálogo público expone los datos bancarios de cada tienda
2026-11-02	F1-25	Hecho	Referencia de Pago Móvil numérica de 4 a 10 dígitos (backend y UI)
2026-11-02	F1-26	Hecho	Cliente puede cancelar pedidos `AWAITING_PAYMENT` y `PENDING`
2026-11-02	F1-27	Hecho	Catálogo, detalle y checkout rechazan tiendas inactivas
2026-11-02	F1-28	Hecho	Solo `CLIENT` puede crear pedidos; admin/superadmin reciben 403
2026-11-02	F1-29	Hecho	`User.email` y `User.phone` dejaron de ser únicos globalmente
2026-11-02	F1-30	Hecho	`@@unique([store_id, email])` y `@@unique([store_id, phone])`; un mismo dato puede repetirse en tiendas distintas
2026-11-02	F1-31	Hecho	`checkEmail` y `verifyOtp` exigen `store_id`, validan tienda activa y buscan el usuario por tienda

Fecha	Fase / ID	Estado	Notas
2026-11-02	F1-32	Hecho	Consultas actualizadas a `findFirst({ email, store_id })` y updates por `id`; `VerificationCode` ahora incluye `store_id`
2026-11-02	F1-33	Hecho	Migración `20261103022000_user_identity_per_store`; respaldo previo de usuarios y códigos OTP
2026-11-02	F1-34	Hecho	Prueba: mismo correo y teléfono en dos tiendas, compra en tienda 2 y pedidos aislados por usuario
2026-11-02	F1-29 extra	Hecho	Transacción de `placeOrder` con `timeout: 20000` y `maxWait: 10000`; P2028 responde 503 en vez de 500

Estados permitidos: `Pendiente`, `En curso`, `Bloqueado`, `Hecho`.

---

11. Decisiones abiertas

Tema	Estado	Cuándo se resuelve
Facturación fiscal	Abierta	Semana 1, con contador
Recetas y lotes	Por validar	Discovery con 5 farmacias, semanas 2–3
Candidatos del piloto	Sin candidatos	Prospección desde semana 1
Zonas y tarifas de delivery	En alcance	Fase 1
Expansión a otra ciudad	Futuro	Después del piloto

---

12. Punto de continuación

Fase 0 y F1-01–F1-34 completadas. Continuar con **F2-01** (aprobación/rechazo de pagos) y **F2-25** (restaurar stock al cancelar), en ese orden si se va a operar con inventario real.

Decisiones de F1-14

- Se usó `Decimal(12,2)` para dinero y `Decimal(14,6)` para tasas; no centavos enteros.
- Un middleware serializa `Prisma.Decimal` como número en JSON para no romper el contrato del frontend.
- Las tasas se interpretan como unidades de la moneda por 1 USD (`ves_rate` = Bs por USD, `cop_rate` = COP por USD).
- Se creó baseline de migraciones y la migración `20261102120100_money_decimal`.

Decisiones de F1-29–F1-34

- La identidad del cliente es por tienda: `@@unique([store_id, email])` y `@@unique([store_id, phone])`; un mismo correo o teléfono puede existir en tiendas distintas.
- `User.username` de administración sigue siendo único global; el teléfono de admin ya no bloquea la creación de tiendas distintas.
- `VerificationCode` incorporó `store_id` y su único pasó a `@@unique([store_id, email, code])`; los códigos antiguos quedan con `store_id = NULL`.
- `checkEmail` exige `store_id`, valida que la tienda exista y esté activa, y limita el reenvío por tienda + correo.
- `verifyOtp` valida el código contra la tienda, busca/crea al usuario por `store_id + email`, actualiza por `id` y devuelve 409 si el teléfono ya existe dentro de la misma tienda.
- Migración aplicada: `20261103022000_user_identity_per_store`.

- Respaldo previo: `C:\Users\Jesus\AppData\Local\Temp\opencode\users-backup-2026-11-03T02-18-39-374Z.json` (5 usuarios, 2 códigos OTP).
- La transacción de `placeOrder` ahora usa `maxWait: 10000` y `timeout: 20000`; si expira (P2028) responde 503 "La operación tardó demasiado. Intenta de nuevo.".

Siguiente bloque recomendado

1. **F2-25:** al cancelar o eliminar un pedido, devolver stock de productos y componentes de combos dentro de una transacción.
2. **F2-01–F2-05:** bandeja `AWAITING_PAYMENT`, aprobar/rechazar pago y bloquear aceptación sin pago aprobado.
3. **F2-06–F2-12:** paginación/filtros de pedidos, ownership en `markOrderViewed` y edición de tienda desde SuperAdmin.
4. **F3:** rate limiting por tienda, auditoría y endurecimiento de sesión/cookies.

Bitácora de la sesión 2026-11-02

- Schema: `User` sin `@unique` en email/teléfono; nuevos únicos compuestos. `VerificationCode.store_id`.
- Código: `authController.js`, `authValidation.js` y `superadminController.js`.
- Pruebas: `auth.api.test.js` cubre mismo correo/teléfono en dos tiendas y compra aislada; `validations.test.js` exige `store_id`.
- DB: migración aplicada con `npx prisma migrate deploy` y `npx prisma generate`.
- Resultados: backend unitario 34/34, API 83/83, frontend 41/41, typecheck y build correctos, lint 0 errores/85 warnings.
- Nota de pruebas: el rate limiter de auth permite ~10 solicitudes/min por IP; reiniciar el servidor entre corridas de `npm run test:api`.
- Documentación: se actualizaron este plan y `Guia_Migracion_VPS.md`; el PDF consolidado quedó en `Documentacion_TiendaFast.pdf` (61 páginas, generado desde README, Brochure, Documentación Completa, Flujo, Guía VPS y este plan).
- Todo el working tree sigue **sin commit** (no se hizo commit porque no se pidió).

Recordatorio de producción

- Antes de vender: completar F2 (admin), F3 (seguridad), F4 (backups/VPS) y F5 (E2E multi-usuario). El aislamiento de clientes por tienda ya está resuelto.
- F2-25 (restaurar stock al cancelar) debe resolverse antes de operar con inventario real.

Cómo ejecutar las pruebas de API

```
$p = Start-Process -FilePath "node" -ArgumentList "server.js" -WorkingDirectory "backend" -PassThru
# Esperar hasta que http://localhost:3000/api/stores/demo-farmacia/products?limit=1 responda 200
npm run test:api
Stop-Process -Id $p.Id -Force
```

No ejecutar `npm run db:seed` sobre la base actual: borra pedidos, productos y categorías.

El rate limiter de autenticación permite ~10 solicitudes por minuto e IP. Si se repite la suite muy seguido, reiniciar el servidor (el contador vive en memoria) antes de volver a correr `npm run test:api`.

Bloqueos actuales

- Ninguno técnico.
- Facturación fiscal pendiente de contador, no bloquea el piloto gratuito.
- Candidatos de piloto pendientes de prospección, no bloquean el desarrollo.

- CI de pruebas de API pendiente para Fase 5.
- Deuda de lint (85 advertencias) a reducir durante Fases 1–3.

---

13. Riesgos

Riesgo	Mitigación
Mercado pequeño en Puerto Ayacucho	Validar y luego expandir a otra ciudad
Conectividad irregular	Optimizar imágenes, VPS en US East, evitar dependencias pesadas
Farmacias exigen recetas o lotes	Discovery temprano y Fase 2C condicional
Facturación fiscal incierta	Contador en semana 1; piloto gratuito
Un solo desarrollador	Documentar, commits pequeños y este registro
Alcance creciente	Fases cerradas y criterios de terminado
Cobro manual	Límite de clientes hasta automatizar

ANEXO C

Plataforma Delivery Multitenant

Documentación Completa — Guía del Sistema v2.0

1. Resumen Ejecutivo

La Plataforma Delivery Multitenant es una solución tecnológica integral para digitalizar negocios de cualquier rubro: farmacias, restaurantes, supermercados, repuestos, bodegas y más. Permite gestionar tiendas virtuales independientes desde un único sistema central, con panel de administración privado, inventario completo, pedidos en tiempo real y soporte multi-monedas.

Lo que ofrece:

- Tienda virtual profesional con URL única por negocio
- Panel de administración en tiempo real con alertas sonoras y visuales
- Gestión de inventario, categorías, combos y precios
- Sistema de pedidos con flujo completo (creación → aceptación → despacho → entrega)
- Multi-monedas automáticas (USD, VES, COP) con tasas en tiempo real
- Notificaciones instantáneas vía WebSockets
- Mapas interactivos con cálculo de distancia y tiempo estimado
- Gestión de motorizados con integración WhatsApp
- Historial de pedidos con reportes e impresión
- Seguridad robusta: JWT, cookies httpOnly, rate limiting, validación de datos

---

2. Arquitectura del Sistema

2.1 Concepto Multitenant

Un único servidor y base de datos aloja múltiples tiendas. Cada tienda tiene:

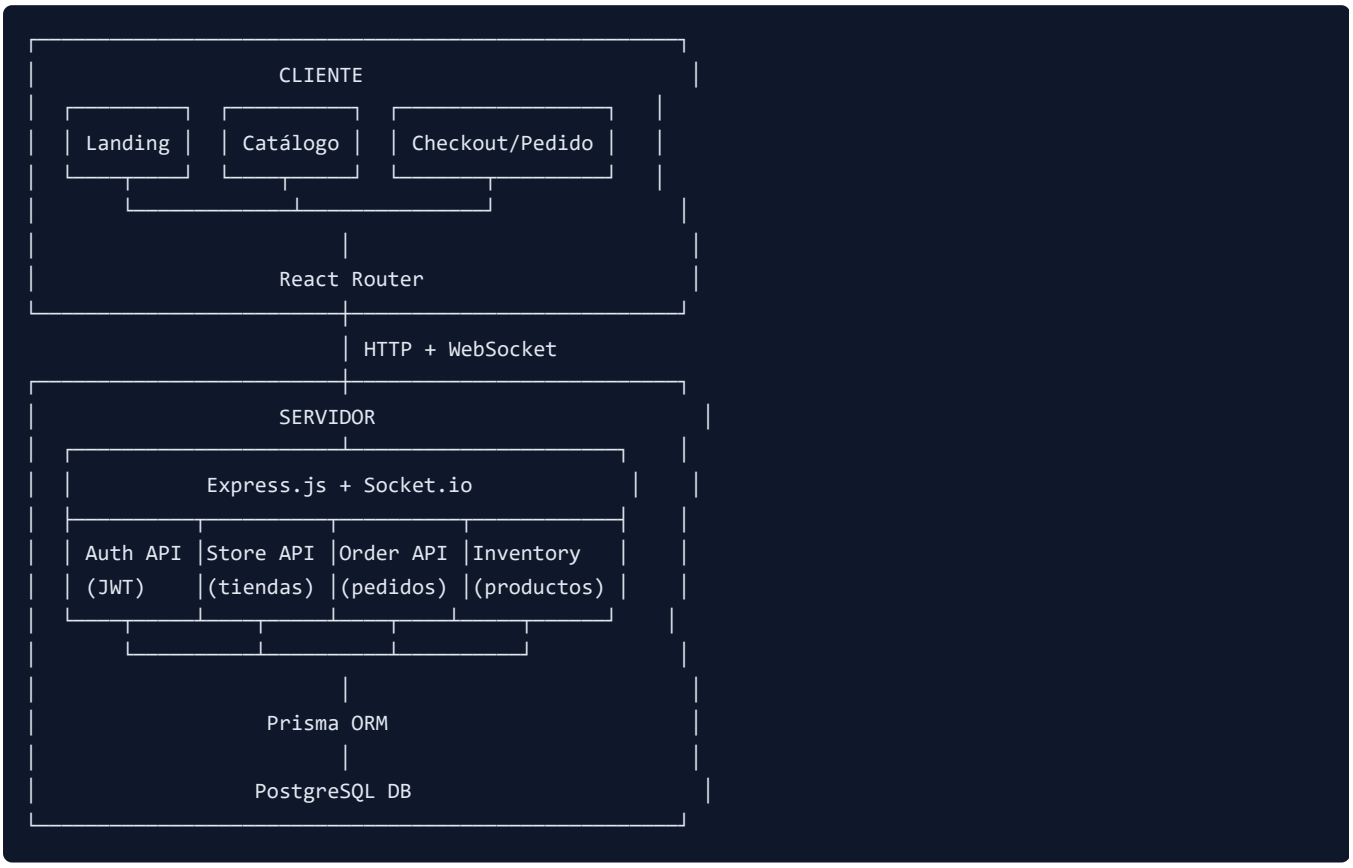
- URL propia: `dominio.com/nombre-tienda`
- Inventario independiente
- Panel de administración aislado
- Configuraciones propias (moneda, ubicación GPS, logo, colores)
- Base de clientes propia

2.2 Stack Tecnológico

Capa	Tecnología	Función
Frontend	React 19 + TypeScript	Interfaz de usuario moderna y tipada
Bundler	Vite	Build ultra rápido para desarrollo y producción
Estilos	TailwindCSS + shadcn/ui	Diseño responsive con componentes profesionales
Mapas	React Leaflet	Mapas interactivos con rutas reales (OSRM)
Gráficas	Recharts	Dashboard con métricas visuales
Animaciones	Framer Motion	Transiciones suaves y micro-animaciones

Capa	Tecnología	Función
Backend	Node.js + Express	API REST de alto rendimiento
Base de datos	PostgreSQL + Prisma ORM	Datos relacionales con integridad referencial
Tiempo real	Socket.io	Comunicación bidireccional instantánea
Almacenamiento	Cloudinary	Imágenes optimizadas en la nube
Email	Resend	Correos transaccionales (códigos OTP)
Autenticación	JWT + bcrypt	Tokens seguros en cookies httpOnly

2.3 Diagrama de Componentes



---

3. Flujo Completo del Cliente

3.1 Descubrimiento y Navegación

Paso 1: Landing Page El cliente accede a `dominio.com/nombre-tienda`. Ve una página de inicio profesional con:

- Banner principal con imagen de la tienda
- Categorías populares con imágenes
- Productos destacados y ofertas
- Sección de contacto y redes sociales
- Botón de WhatsApp flotante

Paso 2: Catálogo de Productos El cliente navega al catálogo completo:

- Barra de búsqueda predictiva (tolera errores tipográficos)
- Filtros por categoría

- Vista de productos en cuadrícula con imagen, nombre y precio
- Precios mostrados en la moneda configurada por la tienda
- Indicador de disponibilidad (agotado vs disponible)

Paso 3: Detalle de Producto Al hacer clic en un producto:

- Imagen en alta calidad
- Nombre, descripción y precio
- Stock disponible
- Botón "Agregar al Carrito"
- Productos relacionados (si existen)

Paso 4: Carrito de Compras

- Lista de productos agregados con cantidades
- Subtotal por producto
- Total con conversión automática de monedas
- Botón para proceder al checkout

3.2 Proceso de Checkout (5 Pasos)

Paso 1: Identificación El cliente ingresa su correo electrónico. El sistema verifica si ya tiene cuenta:

- Si es registrado: muestra sus datos (nombre, teléfono)
- Si es nuevo: pide completar nombre y teléfono

Paso 2: Verificación OTP Se envía un código de 6 dígitos al correo electrónico:

- Código válido por 10 minutos
- Rate limiting: 1 código por minuto por email
- El cliente ingresa el código para autenticarse

Paso 3: Dirección de Entrega

- Buscador de direcciones con autocompletado
- Mapa interactivo para ajustar la ubicación exacta
- Botón "Usar mi ubicación" (GPS del celular)
- Campo para referencias adicionales (piso, casa, etc.)

Paso 4: Método de Pago Dos opciones:

- **Efectivo:** Selección de moneda (USD, VES, COP) con tasas del día
- **Pago Móvil:** Datos bancarios de la tienda + campo para ingresar referencia de 6 dígitos

Paso 5: Confirmación en Tiempo Real

- Timeline animado que muestra el estado del pedido
- El cliente ve cuando el admin acepta, prepara y despacha
- Notificaciones push si otorga permiso

3.3 Seguimiento del Pedido

El cliente puede ver sus pedidos en `dominio.com/tienda/mis-pedidos`:

- Lista cronológica de todos sus pedidos
- Estado actual con colores (Pendiente = naranja, Preparando = azul, En Camino = púrpura, Entregado = verde)
- Timeline detallado con actualizaciones en tiempo real

- Opción de cancelar pedidos pendientes
- Opción de calificar pedidos entregados (1-5 estrellas + reseña)
- Contactar motorizado asignado (si aplica)

---

4. Flujo Completo del Administrador

4.1 Acceso al Panel

URL: `dominio.com/admin-login`

El administrador ingresa con:

- Usuario (username)
- Contraseña (encriptada con bcrypt)

Características de seguridad:

- JWT en cookie httpOnly (no expuesto al JavaScript)
- Token de admin dura 1 día (se renueva automáticamente)
- Bloqueo de cuenta después de 5 intentos fallidos (15 minutos)
- Rate limiting: máximo 5 intentos de login por minuto por IP
- Headers de seguridad HTTP (Helmet)

4.2 Panel de Despachos (Dashboard)

URL: `dominio.com/admin/tienda-slug`

El dashboard es el centro de control principal. Muestra 4 columnas/tabs:

Tab 1: Por Aprobar (Amarillo)

Pedidos con pago por transferencia que necesitan validación:

- El admin ve los datos del pago (referencia, monto)
- Puede aprobar el pago (cambia a PENDIENTE) o rechazarlo
- Ideal para verificar pagos móviles antes de procesar

Tab 2: Nuevas Órdenes (Naranja)

Pedidos en efectivo o pagos aprobados que esperan aceptación:

- Alerta sonora cuando llega un pedido nuevo
- Modal emergente con detalles del pedido (artículos, dirección, monto)
- Botones de Aprobar y Rechazar directamente desde el modal
- Si hay múltiples pedidos nuevos, se muestran en cola secuencialmente
- Indicador visual de pedidos sin leer (dot pulsante)
- Indicador de urgencia para pedidos >10 minutos sin atención

Tab 3: Preparando (Azul)

Pedidos aceptados que la tienda está preparando:

- Lista de pedidos en proceso
- Opción de despachar con motorizado asignado
- Tiempo estimado de entrega visible

Tab 4: En Camino (Púrpura)

Pedidos despachados con motorizado en ruta:

- Información del motorizado asignado
- Opción de marcar como entregado
- Notificación al cliente por WhatsApp (opcional)

Funcionalidades del Dashboard

Alertas de Nuevos Pedidos:

- Sonido de notificación al recibir pedido nuevo
- Notificación nativa del navegador si la pestaña está en segundo plano
- Modal emergente con resumen del pedido
- Si llegan varios pedidos, se muestran en cola (no se sobreescribe)

Hotkeys de Teclado (Desktop):

- `Enter` → Aprobar el pedido actual
- `R` → Abrir dialog de rechazo
- `Escape` → Omitir el pedido actual (siguiente en la cola)

Acciones Rápidas:

- Botones de Aprobar/Rechazar directamente en cada fila (desktop) o tarjeta (móvil)
- No es necesario abrir el detalle del pedido para aceptar o rechazar

Selección Múltiple:

- Botón "Seleccionar" para activar modo selección
- Checkboxes en cada pedido para seleccionar varios
- Botón "Seleccionar todos" para marcar toda la página
- Barra de acciones batch: "Aceptar seleccionados" / "Rechazar seleccionados"

Indicadores Visuales:

- Pedidos sin leer: dot pulsante al lado del ID
- Pedidos urgentes (>10 min): borde rojo + badge "URGENTE"
- Tiempo transcurrido actualizado cada 30 segundos ("Hace 5m")
- Badge de conexión WebSocket (verde = conectado, amarillo = reconectando)

4.3 Detalle de Pedido

URL: `dominio.com/admin/tienda-slug/order/ID`

Vista completa de un pedido con:

Mapa y Ruta:

- Mapa interactivo con marcador de tienda y cliente
- Ruta real calculada por OSRM (Open Source Routing Machine)
- Distancia en kilómetros y tiempo estimado

Datos del Cliente:

- Nombre y teléfono (clickeable para llamar)
- Dirección de entrega completa
- Método de pago y referencia (si aplica)

Lista de Artículos:

- Productos con cantidad, precio unitario y subtotal

- Total del pedido

Acciones según Estado:

Estado Actual	Acción Disponible	Resultado
AWAITING_PAYMENT	Aprobar Pago	Cambia a PENDING
AWAITING_PAYMENT	Rechazar	Cambia a CANCELLED
PENDING	Aceptar	Cambia a ACCEPTED
PENDING	Rechazar	Cambia a CANCELLED
ACCEPTED	Despachar	Abre selector de motorizado
DISPATCHED	Marcar Entregado	Cambia a DELIVERED

Selector de Motorizado:

- Lista de motorizados activos de la tienda
- Opción de omitir (despachar sin motorizado asignado)
- Al despachar: se envía WhatsApp al motorizado con datos del cliente y link de GPS

4.4 Gestión de Inventario

URL: `dominio.com/admin/tienda-slug/inventory`

Panel completo para gestionar productos:

Pestaña de Productos:

- Lista paginada con búsqueda en tiempo real
- Crear nuevo producto: nombre, precio, descripción, imagen, categoría, stock
- Editar producto existente
- Activar/desactivar disponibilidad
- Carga masiva via CSV/Excel (hundreds de productos a la vez)

Pestaña de Categorías:

- Crear, editar y eliminar categorías
- Imagen de categoría para el catálogo público
- Eliminar categoría elimina en cascada sus productos

Pestaña de Combos:

- Crear combos seleccionando productos existentes
- Definir cantidad de cada producto en el combo
- Precio especial del combo
- Stock del combo calculado automáticamente

Características:

- Búsqueda predictiva (fuzzy search)
- Filtrado por categoría
- Paginación server-side (nunca carga todo de golpe)
- Subida de imágenes a Cloudinary (formato WebP automático)

4.5 Gestión de Clientes

URL: `dominio.com/admin/tienda-slug/customers`

- Lista de todos los clientes que han hecho pedidos
- Datos: nombre, email, teléfono, cantidad de pedidos
- Editar información del cliente
- Eliminar cliente (con confirmación)

4.6 Gestión de Motorizados

URL: `dominio.com/admin/tienda-slug/drivers`

- Crear motorizado: nombre, teléfono, placa del vehículo
- Activar/desactivar motorizado
- Editar información
- Eliminar motorizado
- Los motorizados activos aparecen en el selector al despachar pedidos

4.7 Configuración de la Tienda

URL: `dominio.com/admin/tienda-slug/settings`

- **Nombre de la tienda**
- **Moneda principal:** USD, VES o COP
- **Tasas de cambio:** Manual o automática (API del BCV)
- **Ubicación GPS:** Mapa interactivo para fijar la ubicación exacta
- **Industria:** Farmacia, Restaurante, Supermercado, etc.
- **Color del tema:** Personalización visual

4.8 Historial de Pedidos

URL: `dominio.com/admin/tienda-slug/history`

- Lista de pedidos entregados y cancelados
- Filtros por rango de fechas
- KPIs: Ventas de Hoy, Ventas del Mes, Histórico Total, Pedidos Entregados
- Opción de imprimir reporte
- Eliminar pedidos del historial (con confirmación)

---

5. Sistema de Notificaciones en Tiempo Real

5.1 Para el Administrador

Evento	Notificación
Nuevo pedido	Sonido + modal emergente + notificación nativa del SO
Pedido actualizado	Se refleja instantáneamente en el dashboard
Múltiples pedidos	Cola secuencial con indicador de pendientes
Conexión perdida	Indicador "Reconectando..." + reconexión automática

5.2 Para el Cliente

Evento	Notificación
Pedido aceptado	Timeline se actualiza en tiempo real

Evento	Notificación
Pedido en camino	Se muestra datos del motorizado
Pedido entregado	Timeline completa + opción de calificar

5.3 Tecnología WebSocket

- Comunicación bidireccional sin recargar página
- Salas por tienda (`store_{id}`) y por cliente (`client_{id}`)
- Reconexión automática con backoff exponencial
- Fallback a polling HTTP cada 60 segundos si WebSocket falla

---

6. Sistema de Seguridad

6.1 Autenticación

Medida	Implementación
JWT	Tokens firmados con secret de 128 caracteres
Cookies	httpOnly, secure, sameSite: None
Clientes	OTP por email (código de 6 dígitos, expira en 10 min)
Admins	Username + password (bcrypt hash)
Renovación	Token se renueva automáticamente cada 30 minutos
Logout	Limpieza de cookie en servidor y estado en cliente

6.2 Autorización

Rol	Permisos
CLIENT	Crear pedidos, ver sus pedidos, cancelar/calificar
ADMIN	Gestionar su tienda completa (inventario, pedidos, clientes, drivers)
SUPERADMIN	Todo lo de ADMIN + crear tiendas, activar/desactivar tiendas

6.3 Protección de Datos

- **Rate Limiting:** Límite de peticiones por IP y por ruta
- **Body Size Limit:** Máximo 1MB por request
- **Helmet:** Headers de seguridad HTTP (HSTS, CSP, X-Frame-Options)
- **CORS:** Lista blanca de dominios permitidos
- **Validación Zod:** Todos los inputs validados con schemas tipados
- **Precios server-side:** El backend recalcula precios desde la DB (nunca confía en el cliente)
- **IDOR Protection:** Middlewares que verifican que cada admin solo acceda a su tienda
- **Contra carrera:** Updates con condición WHERE para evitar conflictos multi-admin
- **Cuenta bloqueada:** 5 intentos fallidos → bloqueo de 15 minutos

6.4 Infraestructura

- Base de datos PostgreSQL en Neon (cloud)
- Backend en Render (auto-scaling)

- Frontend en Vercel (CDN global)
- Imágenes en Cloudinary (optimización automática WebP)

---

7. Gestión de Monedas

7.1 Configuración

Cada tienda define su moneda principal y tasas de cambio:

- **USD** (Dólares Americanos)
- **VES** (Bolívares Venezolanos)
- **COP** (Pesos Colombianos)

7.2 Conversión Automática

- Tasas actualizadas manualmente o vía API del BCV
- El cliente ve precios en la moneda de la tienda
- En el checkout, se muestra el desglose en todas las monedas
- El admin cobra en la moneda que el cliente elija

7.3 Ejemplo

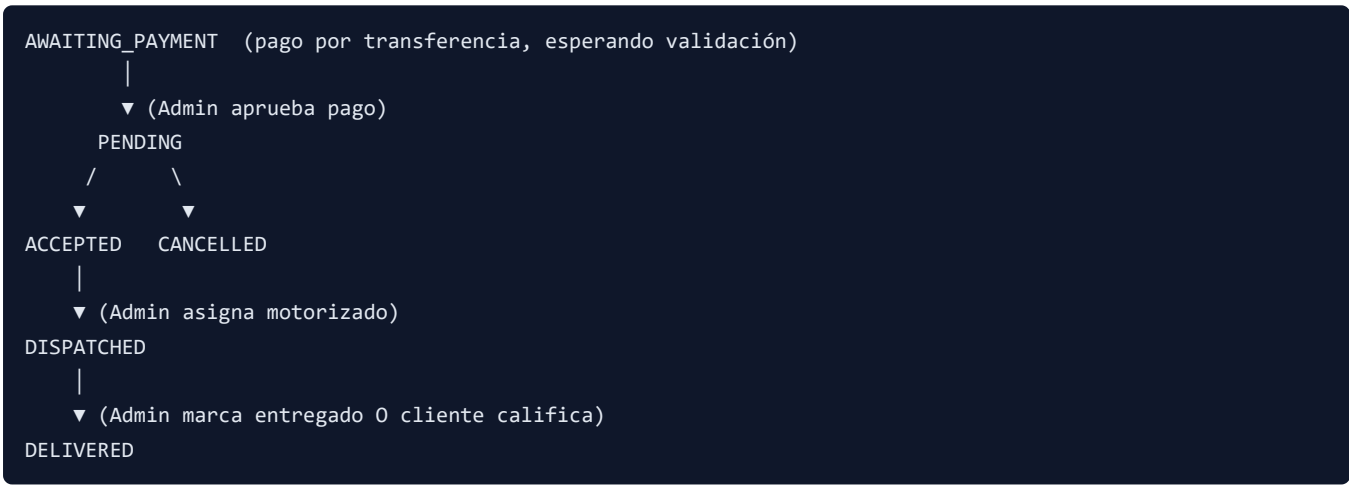
Tienda configurada en USD con tasa VES = 36.5:

- Producto: $10.00 USD
- Cliente ve: Bs. 365.00 VES
- En checkout: "Total a pagar: $10.00 USD / Bs. 365.00 VES"

---

8. Gestión de Pedidos — Estado y Flujo

8.1 Estados del Pedido



8.2 Transiciones Válidas

Desde	Hasta	Quién
AWAITING_PAYMENT	PENDING	Admin (aprueba pago)
AWAITING_PAYMENT	CANCELLED	Admin (rechaza pago)

Desde	Hasta	Quién
PENDING	ACCEPTED	Admin (acepta pedido)
PENDING	CANCELLED	Admin o Cliente
ACCEPTED	DISPATCHED	Admin (despacha)
ACCEPTED	CANCELLED	Admin
DISPATCHED	DELIVERED	Admin o Cliente (califica)

8.3 Protección Contra Carrera

Si dos admins intentan modificar el mismo pedido simultáneamente:

- El primer cambio se aplica
- El segundo recibe error 409 "Pedido ya fue modificado"
- El dashboard recarga automáticamente la lista

---

9. Panel de Super Administrador

URL: `dominio.com/superadmin`

Acceso exclusivo para el propietario de la plataforma:

- **Crear tiendas:** Registra una nueva tienda con su admin asociado
- **Activar/Desactivar tiendas:** Control de acceso por tienda
- **Listar todas las tiendas:** Vista general de todas las tiendas del sistema
- **Acceso a cualquier tienda:** Puede ver y gestionar cualquier tienda

---

10. Funcionalidades Técnicas Avanzadas

10.1 Cálculo de Distancia y Tiempo

- Fórmula de Haversine para calcular distancia entre tienda y cliente
- Factor de corrección x1.4 para distancia real (no recta)
- Tiempo estimado: `(distancia_km / 25) * 60 + 15` minutos
- Ruta real calculada por OSRM (Open Source Routing Machine)

10.2 Carga Masiva de Productos

- Subir archivo CSV o Excel con productos
- Validación de columnas requeridas
- Creación en lote con `createMany` de Prisma
- Manejo de errores por fila

10.3 Integración WhatsApp

- Al despachar pedido: envía WhatsApp al motorizado con:
  - Datos del cliente (nombre, teléfono)
  - Dirección de entrega
  - Link de navegación GPS
- Opcionalmente notifica al cliente con datos del motorizado

10.4 Impresión de Reportes

- Historial de pedidos con filtro por fechas
- KPIs de ventas (hoy, mes, histórico)
- Botón de imprimir con estilos optimizados para papel

---

11. Experiencia de Usuario

11.1 Diseño Visual

- **Glassmorphism:** Elementos translúcidos con desenfoques de fondo
- **Micro-animaciones:** Transiciones suaves entre páginas
- **Fuentes:** Manrope (cuerpo) + Sora (encabezados)
- **Colores:** Tema personalizable por tienda

11.2 Responsive

- **Desktop:** Tabla completa con todas las columnas
- **Móvil:** Tarjetas verticales con acciones rápidas
- **Tablet:** Layout adaptativo entre ambos

11.3 Accesibilidad

- Contraste de colores WCAG AA
- Navegación por teclado (hotkeys en admin)
- Labels en formularios
- Estados visuales claros (colores + iconos)

---

12. Requisitos para Implementación

12.1 Para el Cliente (Tienda)

Requisito	Descripción
Dominio	Un nombre de dominio (ej: `mitienda.com`)
Logo	Imagen de la tienda para la landing page
Productos	Lista de productos con nombre, precio, descripción e imagen
Categorías	Clasificación de productos (mínimo 1)
Ubicación GPS	Coordenadas exactas de la tienda
Datos bancarios	Para pagos móviles (opcional)
Número WhatsApp	Para notificaciones a motorizados (opcional)

12.2 Para el Proveedor (Nosotros)

Requisito	Descripción
Servidor	Render (ya configurado)
Base de datos	PostgreSQL en Neon (ya configurado)
Cloudinary	Cuenta para almacenamiento de imágenes

Requisito	Descripción
Dominio DNS	Configuración de subdominios
SSL	Certificado HTTPS (automático en Vercel/Render)

---

13. Roadmap de Mejoras Futuras

Fase 3 (Próximamente)

- Notificaciones push móviles (PWA)
- App móvil nativa para administradores
- Sistema de suscripciones y planes
- Múltiples motorizados por pedido
- Chat en tiempo real cliente-admin
- Sistema de devoluciones
- Cupones y descuentos
- Programa de fidelidad

Fase 4 (Escalabilidad)

- Subdominios por tienda (`farmacia.dominio.com`)
- Branding completo por tienda (logo, colores, fuentes)
- API pública para integraciones externas
- Webhooks para sistemas de inventario
- Analytics avanzados con ML
- Multi-idioma (i18n)

---

14. Soporte y Mantenimiento

14.1 Actualizaciones

- El sistema se actualiza sin downtime (deploy continuo)
- Frontend: Vercel (deploy automático al hacer push)
- Backend: Render (deploy automático al hacer push)

14.2 Backups

- Base de datos: Neon realiza backups automáticos diarios
- Imágenes: Cloudinary mantiene original + backups

14.3 Monitoreo

- Logs de errores en el servidor
- Métricas de rendimiento
- Alertas de caída del servicio

---

15. Preguntas Frecuentes

¿Puedo tener más de una tienda? Sí. La arquitectura multitenant permite crear infinitas tiendas desde el panel de Super Admin.

¿Necesito saber programar? No. El panel de administración es 100% visual. Puedes gestionar productos, pedidos, clientes y configuraciones sin tocar código.

¿Funciona en el celular? Sí. Tanto la tienda pública como el panel de administración son 100% responsive y funcionan en cualquier dispositivo.

¿Cuántos productos puedo cargar? No hay límite. El sistema usa paginación inteligente que carga productos de forma gradual.

¿Cómo se actualizan las tasas de cambio? Puedes configurarlas manualmente en ajustes o conectar la API del BCV para actualización automática.

¿Puedo personalizar los colores de mi tienda? Sí. Cada tienda tiene su propio color de tema configurable desde el panel de administración.

¿Los pedidos se pierden si se cae internet? No. Los pedidos se guardan en la base de datos. Si la conexión se restaura, todo se sincroniza automáticamente.

¿Puedo tener varios admins en la misma tienda? Sí. Cada tienda puede tener múltiples administradores con las mismas credenciales o credenciales separadas.

Documento generado para la Plataforma Delivery Multitenant v2.0 Última actualización: Agosto 2026

ANEXO D

Flujo Completo de la Aplicacion — Plataforma Delivery Multitenant

1. Vista General

La Plataforma Delivery Multitenant es un sistema completo de ventas y gestion de inventario para negocios de cualquier rubro (farmacias, restaurantes, supermercados, repuestos, bodegas). Permite crear multiples tiendas virtuales independientes desde un unico servidor, cada una con su URL propia, inventario, panel de administracion y base de clientes.

URL de produccion: `https://test.shop-mg.com` **Backend API:** `https://delivery-app-3ewq.onrender.com/api`

2. Arquitectura del Sistema

2.1 Componentes Principales

Componente	Tecnologia	Funcion
Frontend	React 19 + TypeScript + Vite 8	Interfaz de usuario moderna y responsive
Backend	Node.js + Express.js	API REST y WebSockets
Base de datos	PostgreSQL + Prisma ORM	Almacenamiento relacional
Tiempo real	Socket.io	Notificaciones instantaneas
Imagenes	Cloudinary	Almacenamiento optimizado en la nube
Email	Resend	Envio de codigos OTP
Mapas	OSRM + Leaflet	Rutas reales y mapas interactivos
Auth	JWT + bcrypt	Autenticacion segura con cookies httpOnly

2.2 Arquitectura Multitenant

Cada tienda opera de forma completamente aislada:

- URL propia: `dominio.com/nombre-tienda`
- Inventario independiente
- Panel de administracion privado
- Configuraciones propias (moneda, ubicacion GPS, logo, colores)
- Base de clientes propia
- Multiples administradores por tienda

2.3 Diagrama de Flujo de Datos



---

3. Flujo del Cliente (Comprador)

3.1 Acceso a la Tienda

El cliente accede a la URL de la tienda:

```
https://test.shop-mg.com/farmacia-ayacucho
```

Lo que ve:

- Landing page profesional con banner principal
- Categorias populares con imagenes destacadas
- Productos destacados y ofertas
- Seccion de contacto y redes sociales
- Boton de WhatsApp flotante
- Barra de navegacion con buscador, carrito y pedidos

3.2 Navegacion del Catalogo

Explorar categorias:

- Haz clic en una categoria para ver solo sus productos
- Cada categoria tiene imagen y nombre descriptivo

Buscar productos:

- Usa la barra de busqueda en la parte superior
- Busqueda predictiva que tolera errores tipographicos

- Muestra resultados en tiempo real mientras escribes

Ver producto:

- Haz clic en cualquier producto del catalogo
- Ve imagen en alta calidad, nombre, descripcion, precio y stock disponible
- Selecciona cantidad y haz clic en "Agregar al Carrito"

3.3 Carrito de Compras

Agregar productos:

- Desde el catalogo o desde el detalle del producto
- El icono del carrito muestra la cantidad de items

Ver carrito:

- Haz clic en el icono del carrito en la barra superior
- Lista de productos con cantidad, precio unitario y subtotal
- Total calculado automaticamente en la moneda de la tienda
- Boton para eliminar productos
- Boton para proceder al checkout

3.4 Proceso de Checkout (5 Pasos)

Paso 1: Identificacion

- Ingresa tu correo electronico
- Si es la primera vez: completa nombre y telefono
- Si ya tienes cuenta: se cargan tus datos automaticamente

Paso 2: Verificacion OTP

- Se envia un codigo de 6 digitos a tu correo
- Ingresa el codigo en el campo correspondiente
- El codigo es valido por 10 minutos
- Rate limiting: 1 codigo por minuto por email

Paso 3: Direccion de Entrega

- Escribe tu direccion en el buscador
- Se autocompleta con sugerencias
- Ajusta la ubicacion exacta arrastrando el pin en el mapa
- O usa "Mi ubicacion" para GPS automatico
- Campo opcional para referencias (piso, casa, edificio, etc.)

Paso 4: Metodo de Pago

Dos opciones disponibles:

Efectivo:

- Selecciona la moneda (USD, VES o COP)
- El sistema muestra la tasa de cambio del dia
- Indica si necesitas cambio

Pago Movil:

- Se muestran los datos bancarios de la tienda

- Ingresa la referencia de 6 digitos de tu transferencia
- El pedido queda en estado "Por Aprobar" hasta que el admin valide

Paso 5: Confirmacion

- Revisa el resumen del pedido
- Haz clic en "Realizar Pedido"
- El pedido se envia al administrador en tiempo real

3.5 Seguimiento del Pedido

Acceder a mis pedidos:

- Ve a `https://test.shop-mg.com/farmacia-ayacucho/mis-pedidos`
- Requiere estar autenticado (login con OTP)

Lo que ves:

- Lista cronologica de todos tus pedidos
- Estado actual con colores:
  - Naranja: Pendiente (esperando aceptacion)
  - Azul: Preparando
  - Purpura: En Camino (con motorizado)
  - Verde: Entregado
  - Rojo: Cancelado
- Timeline detallado con actualizaciones en tiempo real
- Monto total pagado
- Direccion de entrega

Acciones disponibles:

Estado	Accion
Pendiente	Cancelar pedido
Entregado	Calificar con 1-5 estrellas + resena

3.6 Calificar un Pedido

Despues de que el pedido se marca como entregado:

- Haz clic en "Calificar"
- Selecciona de 1 a 5 estrellas
- Escribe una resena (opcional)
- Confirma la calificacion

---

4. Flujo del Administrador (Dueño de la Tienda)

4.1 Inicio de Sesion

URL: `https://test.shop-mg.com/admin-login`

Credenciales:

- Usuario: username del admin
- Contraseña: password encriptada

Seguridad:

- JWT en cookie httpOnly (no expuesto al JavaScript)
- Bloqueo de cuenta despues de 5 intentos fallidos (15 minutos)
- Rate limiting: maximo 5 intentos por minuto por IP
- Token de admin dura 1 dia (se renueva automaticamente cada 30 minutos)

4.2 Panel de Despachos (Dashboard Principal)

URL: `https://test.shop-mg.com/admin/farmacia-ayacucho`

El dashboard es el centro de control. Muestra 4 columnas/tabs:

Tab 1: Por Aprobar (Amarillo)

Pedidos con pago por transferencia que necesitan validacion:

- El admin ve los datos del pago (referencia, monto, banco)
- Puede aprobar el pago (cambia a PENDIENTE) o rechazarlo
- Ideal para verificar pagos moviles antes de procesar

Tab 2: Nuevas Ordenes (Naranja)

Pedidos en efectivo o pagos aprobados que esperan aceptacion:

- Alerta sonora cuando llega un pedido nuevo
- Modal emergente con detalles del pedido (articulos, direccion, monto)
- Botones de Aprobar y Rechazar directamente desde el modal
- Si hay multiples pedidos nuevos, se muestran en cola secuencialmente
- Indicador visual de pedidos sin leer (dot pulsante)
- Indicador de urgencia para pedidos de mas de 10 minutos sin atencion

Tab 3: Preparando (Azul)

Pedidos aceptados que la tienda esta preparando:

- Lista de pedidos en proceso
- Opcion de despachar con motorizado asignado
- Tiempo estimado de entrega visible

Tab 4: En Camino (Purpura)

Pedidos despachados con motorizado en ruta:

- Informacion del motorizado asignado
- Opcion de marcar como entregado
- Notificacion al cliente por WhatsApp (opcional)

4.3 Funcionalidades del Dashboard

Alertas de Nuevos Pedidos

- Sonido de notificacion al recibir pedido nuevo
- Notificacion nativa del navegador si la pestana esta en segundo plano
- Modal emergente con resumen del pedido
- Si llegan varios pedidos, se muestran en cola (no se sobreescribe)

Hotkeys de Teclado (Desktop)

- `Enter` → Aprobar el pedido actual

- `R` → Abrir dialog de rechazo
- `Escape` → Omitir el pedido actual (siguiente en la cola)

Acciones Rapidas

- Botones de Aprobar/Rechazar directamente en cada fila (desktop) o tarjeta (movil)
- No es necesario abrir el detalle del pedido para aceptar o rechazar

Seleccion Multiple

- Boton "Seleccionar" para activar modo seleccion
- Checkboxes en cada pedido para seleccionar varios
- Boton "Seleccionar todos" para marcar toda la pagina
- Barra de acciones batch: "Aceptar seleccionados" / "Rechazar seleccionados"

Indicadores Visuales

- Pedidos sin leer: dot pulsante al lado del ID
- Pedidos urgentes (>10 min): borde rojo + badge "URGENTE"
- Tiempo transcurrido actualizado cada 30 segundos ("Hace 5m")
- Badge de conexion WebSocket (verde = conectado, amarillo = reconectando)

4.4 Detalle de Pedido

URL: `https://test.shop-mg.com/admin/farmacia-ayacucho/order/ID`

Vista completa de un pedido:

Mapa y Ruta:

- Mapa interactivo con marcador de tienda y cliente
- Ruta real calculada por OSRM (Open Source Routing Machine)
- Distancia en kilometros y tiempo estimado

Datos del Cliente:

- Nombre y telefono (clickeable para llamar)
- Direccion de entrega completa
- Metodo de pago y referencia (si aplica)

Lista de Articulos:

- Productos con cantidad, precio unitario y subtotal
- Total del pedido

Acciones segun Estado:

Estado Actual	Accion Disponible	Resultado
AWAITING_PAYMENT	Aprobar Pago	Cambia a PENDING
AWAITING_PAYMENT	Rechazar	Cambia a CANCELLED
PENDING	Aceptar	Cambia a ACCEPTED
PENDING	Rechazar	Cambia a CANCELLED
ACCEPTED	Despachar	Abre selector de motorizado
DISPATCHED	Marcar Entregado	Cambia a DELIVERED

Selector de Motorizado:

- Lista de motorizados activos de la tienda
- Opcion de omitir (despachar sin motorizado asignado)
- Al despachar: se envia WhatsApp al motorizado con datos del cliente y link de GPS

4.5 Gestion de Inventario

URL: `https://test.shop-mg.com/admin/farmacia-ayacucho/inventory`

Panel completo para gestionar productos:

Pestana de Productos:

- Lista paginada con busqueda en tiempo real
- Crear nuevo producto: nombre, precio, descripcion, imagen, categoria, stock
- Editar producto existente
- Activar/desactivar disponibilidad
- Carga masiva via CSV/Excel (cientos de productos a la vez)

Pestana de Categorias:

- Crear, editar y eliminar categorias
- Imagen de categoria para el catalogo publico
- Eliminar categoria elimina en cascada sus productos

Pestana de Combos:

- Crear combos seleccionando productos existentes
- Definir cantidad de cada producto en el combo
- Precio especial del combo
- Stock del combo calculado automaticamente

Caracteristicas:

- Busqueda predictiva (fuzzy search)
- Filtrado por categoria
- Paginacion server-side (nunca carga todo de golpe)
- Subida de imagenes a Cloudinary (formato WebP automatico)

4.6 Gestion de Clientes

URL: `https://test.shop-mg.com/admin/farmacia-ayacucho/customers`

- Lista de todos los clientes que han hecho pedidos
- Datos: nombre, email, telefono, cantidad de pedidos
- Editar informacion del cliente
- Eliminar cliente (con confirmacion)

4.7 Gestion de Motorizados

URL: `https://test.shop-mg.com/admin/farmacia-ayacucho/drivers`

- Crear motorizado: nombre, telefono, placa del vehiculo
- Activar/desactivar motorizado
- Editar informacion
- Eliminar motorizado
- Los motorizados activos aparecen en el selector al despachar pedidos

4.8 Configuracion de la Tienda

URL: `https://test.shop-mg.com/admin/farmacia-ayacucho/settings`

- **Nombre de la tienda**
- **Moneda principal:** USD, VES o COP
- **Tasas de cambio:** Manual o automatica (API del BCV)
- **Ubicacion GPS:** Mapa interactivo para fijar la ubicacion exacta
- **Industria:** Farmacia, Restaurante, Supermercado, etc.
- **Color del tema:** Personalizacion visual

4.9 Historial de Pedidos

URL: `https://test.shop-mg.com/admin/farmacia-ayacucho/history`

- Lista de pedidos entregados y cancelados
- Filtros por rango de fechas
- KPIs: Ventas de Hoy, Ventas del Mes, Historico Total, Pedidos Entregados
- Opcion de imprimir reporte
- Eliminar pedidos del historial (con confirmacion)

---

5. Estados del Pedido

5.1 Diagrama de Estados

```
AWAITING_PAYMENT (pago por transferencia, esperando validacion)
        |
        ▼ (Admin aprueba pago)
      PENDING
     /      \
    ▼        ▼
ACCEPTED   CANCELLED
    |
    ▼ (Admin asigna motorizado)
DISPATCHED
    |
    ▼ (Admin marca entregado O cliente califica)
DELIVERED
```

5.2 Tabla de Transiciones

Desde	Hasta	Quien la ejecuta
AWAITING_PAYMENT	PENDING	Admin (aprueba pago)
AWAITING_PAYMENT	CANCELLED	Admin (rechaza pago)
PENDING	ACCEPTED	Admin (acepta pedido)
PENDING	CANCELLED	Admin o Cliente
ACCEPTED	DISPATCHED	Admin (despacha)
ACCEPTED	CANCELLED	Admin
DISPATCHED	DELIVERED	Admin o Cliente (califica)

5.3 Proteccion Contra Carrera

Si dos admins intentan modificar el mismo pedido simultaneamente:

- El primer cambio se aplica
- El segundo recibe error 409 "Pedido ya fue modificado"
- El dashboard recarga automaticamente la lista

---

6. Sistema de Notificaciones en Tiempo Real

6.1 Para el Administrador

Evento	Notificacion
Nuevo pedido	Sonido + modal emergente + notificacion nativa del SO
Pedido actualizado	Se refleja instantaneamente en el dashboard
Multiples pedidos	Cola secuencial con indicador de pendientes
Conexion perdida	Indicador "Reconectando..." + reconexion automatica

6.2 Para el Cliente

Evento	Notificacion
Pedido aceptado	Timeline se actualiza en tiempo real
Pedido en camino	Se muestra datos del motorizado
Pedido entregado	Timeline completa + opcion de calificar

6.3 Tecnologia WebSocket

- Comunicacion bidireccional sin recargar pagina
- Salas por tienda (`store_{id}`) y por cliente (`client_{id}`)
- Reconexion automatica con backoff exponencial
- Fallback a polling HTTP cada 60 segundos si WebSocket falla

---

7. Sistema de Seguridad

7.1 Autenticacion

Medida	Implementacion
JWT	Tokens firmados con secreto de 128 caracteres
Cookies	httpOnly, secure, sameSite: None
Clientes	OTP por email (codigo de 6 digitos, expira en 10 min)
Admins	Username + password (bcrypt hash)
Renovacion	Token se renueva automaticamente cada 30 minutos
Logout	Limpieza de cookie en servidor y estado en cliente

7.2 Autorizacion

Rol	Permisos
CLIENT	Crear pedidos, ver sus pedidos, cancelar/calificar
ADMIN	Gestionar su tienda completa (inventario, pedidos, clientes, drivers)
SUPERADMIN	Todo lo de ADMIN + crear tiendas, activar/desactivar tiendas

7.3 Proteccion de Datos

- **Rate Limiting:** Limite de peticiones por IP y por ruta
- **Body Size Limit:** Maximo 1MB por request
- **Helmet:** Headers de seguridad HTTP (HSTS, CSP, X-Frame-Options)
- **CORS:** Lista blanca de dominios permitidos
- **Validacion Zod:** Todos los inputs validados con schemas tipados
- **Precios server-side:** El backend recalcula precios desde la DB (nunca confia en el cliente)
- **IDOR Protection:** Middlewares que verifican que cada admin solo acceda a su tienda
- **Contra carrera:** Updates con condicion WHERE para evitar conflictos multi-admin
- **Cuenta bloqueada:** 5 intentos fallidos → bloqueo de 15 minutos

---

8. Gestion de Monedas

8.1 Configuracion

Cada tienda define su moneda principal y tasas de cambio:

- **USD** (Dolares Americanos)
- **VES** (Bolivares Venezolanos)
- **COP** (Pesos Colombianos)

8.2 Conversion Automatica

- Tasas actualizadas manualmente o via API del BCV
- El cliente ve precios en la moneda de la tienda
- En el checkout, se muestra el desglose en todas las monedas
- El admin cobra en la moneda que el cliente elija

8.3 Ejemplo

Tienda configurada en USD con tasa VES = 36.5:

- Producto: $10.00 USD
- Cliente ve: Bs. 365.00 VES
- En checkout: "Total a pagar: $10.00 USD / Bs. 365.00 VES"

---

9. Panel de Super Administrador

URL: `https://test.shop-mg.com/superadmin`

Acceso exclusivo para el propietario de la plataforma:

- **Crear tiendas:** Registra una nueva tienda con su admin asociado
- **Activar/Desactivar tiendas:** Control de acceso por tienda

- **Listar todas las tiendas:** Vista general de todas las tiendas del sistema
- **Acceso a cualquier tienda:** Puede ver y gestionar cualquier tienda

---

10. Funcionalidades Tecnicas Avanzadas

10.1 Calculo de Distancia y Tiempo

- Formula de Haversine para calcular distancia entre tienda y cliente
- Factor de correccion x1.4 para distancia real (no recta)
- Tiempo estimado: `(distancia_km / 25) * 60 + 15` minutos
- Ruta real calculada por OSRM (Open Source Routing Machine)

10.2 Carga Masiva de Productos

- Subir archivo CSV o Excel con productos
- Validacion de columnas requeridas
- Creacion en lote con `createMany` de Prisma
- Manejo de errores por fila

10.3 Integracion WhatsApp

- Al despachar pedido: envia WhatsApp al motorizado con:
  - Datos del cliente (nombre, telefono)
  - Direccion de entrega
  - Link de navegacion GPS
- Opcionalmente notifica al cliente con datos del motorizado

10.4 Impresion de Reportes

- Historial de pedidos con filtro por fechas
- KPIs de ventas (hoy, mes, historico)
- Boton de imprimir con estilos optimizados para papel

---

11. Experiencia de Usuario

11.1 Diseno Visual

- **Glassmorphism:** Elementos translucidos con desenfoques de fondo
- **Micro-animaciones:** Transiciones suaves entre paginas
- **Fuentes:** Manrope (cuerpo) + Sora (encabezados)
- **Colores:** Tema personalizable por tienda

11.2 Responsive

- **Desktop:** Tabla completa con todas las columnas
- **Movil:** Tarjetas verticales con acciones rapidas
- **Tablet:** Layout adaptativo entre ambos

11.3 Accesibilidad

- Contraste de colores WCAG AA

- Navegacion por teclado (hotkeys en admin)
- Labels en formularios
- Estados visuales claros (colores + iconos)

---

12. Resumen de URLs

Cliente

Pagina	URL
Landing	`dominio.com/:slug`
Catalogo	`dominio.com/:slug/productos`
Categoria	`dominio.com/:slug/categorias/:id`
Detalle producto	`dominio.com/:slug/productos/:id`
Checkout	`dominio.com/:slug/checkout`
Mis pedidos	`dominio.com/:slug/mis-pedidos`

Administrador

Pagina	URL
Login	`dominio.com/admin-login`
Dashboard	`dominio.com/admin/:slug`
Detalle pedido	`dominio.com/admin/:slug/order/:id`
Inventario	`dominio.com/admin/:slug/inventory`
Clientes	`dominio.com/admin/:slug/customers`
Motorizados	`dominio.com/admin/:slug/drivers`
Configuracion	`dominio.com/admin/:slug/settings`
Historial	`dominio.com/admin/:slug/history`

Super Admin

Pagina	URL
Panel	`dominio.com/superadmin`

Documento generado para la Plataforma Delivery Multitenant v2.0 Ultima actualizacion: Agosto 2026

ANEXO E

📦 Plataforma Delivery "Multitenant" - Presentación Comercial

Una solución tecnológica completa para digitalizar, administrar y escalar negocios de cualquier rubro (Farmacias, Repuestos, Restaurantes, Bodegas y más).

🚀 ¿Qué es esta plataforma?

Es un sistema de ventas y gestión de inventario de **alto rendimiento**, diseñado bajo una arquitectura **Multitenant**. Esto significa que desde un único sistema central, puedes crear y alojar infinitas tiendas virtuales independientes.

Cada tienda tiene su propia dirección web (ej: `tu-sitio.com/farmacia-ayacucho`), su propio inventario, su panel de control privado y sus propias configuraciones de negocio.

💎 Ventajas Competitivas Únicas

1. Sistema Multi-Moneda Dinámico (En tiempo real)

El mayor problema del comercio actual es la fluctuación de las monedas. Esta plataforma está conectada directamente a las APIs del **Banco Central de Venezuela (BCV)** y **DólarAPI**.

- **Autoprecios:** Puedes guardar tus productos en Dólares (USD) y la plataforma le mostrará al cliente el precio exacto en Bolívares (VES) a la tasa del día, de forma automática.
- **Cobro Inteligente:** El carrito de compras calcula y desglosa el total en **Dólares, Bolívares y Pesos Colombianos**, dándole claridad absoluta al cliente antes de pagar.

2. Sincronización en Tiempo Real (WebSockets)

Olvídate de "recargar la página" para ver si llegó un pedido nuevo.

- **Panel Administrativo Vivo:** Cuando un cliente hace un pedido, el panel de administración del negocio recibe una notificación visual y sonora al instante (0.1 segundos).
- **Gestión Inmediata:** Si el administrador cambia el precio de un producto, marca un ítem como "Agotado" o crea un Combo, todos los clientes que estén viendo la tienda verán la actualización en sus pantallas mágicamente, sin refrescar la página.

3. Rendimiento Extremo y Paginación Inteligente

Tu negocio puede tener 10 productos o **50,000 productos**; la plataforma siempre será rápida.

- **Lazy-Loading:** Desarrollamos un algoritmo de carga perezosa. El servidor nunca carga el catálogo completo, sino que entrega pequeñas fracciones de datos según el usuario navega o busca un producto.
- **Buscador Predictivo (Fuzzy Search):** Un buscador global que tolera errores tipográficos y busca coincidencias parciales a velocidad de la luz.

4. Experiencia de Usuario (UX) Premium

El frontend está construido con las tecnologías más modernas (React + Vite + TailwindCSS), garantizando:

- **Diseño Glassmorphism:** Elementos translúcidos, desenfoques de fondo y sombras suaves que transmiten lujo y modernidad.

- **Micro-Animaciones:** Botones que reaccionan al pasar el ratón, transiciones suaves entre páginas y notificaciones flotantes (Toasts) amigables.
- **Responsive al 100%:** La aplicación se adapta milimétricamente a pantallas de celulares, tablets y computadoras de escritorio.

📱 El Flujo Perfecto

🛍️ Para el Cliente (Comprador)

1. **Entra a la Tienda:** Llega a una página de inicio hermosa con las categorías más populares.
2. **Explora el Catálogo:** Navega entre productos y Combos especiales.
3. **Detalles Claros:** Al hacer clic en un producto, ve una imagen en alta calidad, descripción detallada, stock disponible y precios en todas las monedas.
4. **Carrito Reactivo:** Agrega productos. El carrito suma instantáneamente usando la tasa de cambio del día.
5. **Checkout Sencillo:** Deja sus datos, indica si necesita vuelto y envía la orden. (El sistema envía la orden directamente al administrador sin intermediarios).

🏢 Para el Dueño del Local (Administrador)

1. **Login Seguro:** Accede con credenciales encriptadas (JWT).
2. **Dashboard Financiero:** Ve un resumen de sus ventas y pedidos recientes.
3. **Gestor de Inventario Ultra-Rápido:**
    - Pestañas separadas para Categorías, Productos Individuales y Combos.
    - Creación de Combos seleccionando productos existentes de su almacén para liquidar stock rápidamente.
4. **Configuraciones de su Local:**
    - Define su moneda principal de venta (USD, COP, VES).
    - Ajusta una tasa manual o deja que la API lo haga por él.
    - **GPS Interactivo:** Mueve un Pin en un mapa interactivo para guardar las coordenadas exactas de su local y que el sistema sepa desde dónde salen los deliverys.

🛠️ Tecnologías Empleadas (Para los más técnicos)

- **Frontend:** React.js, TypeScript, TailwindCSS, Lucide Icons, React-Leaflet (Mapas).
- **Backend:** Node.js, Express, Socket.io (Tiempo real).
- **Base de Datos:** PostgreSQL con Prisma ORM (Relaciones complejas e integridad referencial).
- **Almacenamiento Multimedia:** Cloudinary Cloud Storage (Para imágenes ultra optimizadas de los productos).

> **Resumen para Venta:**
> "No te ofrezco un simple catálogo de WhatsApp. Te ofrezco un ecosistema de ventas profesional, que se actualiza solo con la tasa del BCV, avisa al instante cuando compran y hace que tus productos luzcan irresistibles en el celular de tus clientes."

ANEXO F

🚀 Plataforma Delivery Multitenant (Beta v1.0)

Bienvenido al repositorio de la plataforma de Delivery y Gestión de Inventario. Este proyecto está diseñado para alojar múltiples negocios (Farmacias, Repuestos, Comida, etc.) desde una única base de datos y servidor, brindando a cada tienda su propia identidad y panel de administración independiente.

✨ Funcionalidades Principales

- **Arquitectura Multitenant:** Una sola instancia del código maneja infinitas tiendas aisladas mediante URLs únicas (ej: `/farmacia-ayacucho`).
- **Multi-Moneda Dinámico (En tiempo real):** Integración nativa con la API del **Banco Central de Venezuela (BCV)** y **DólarAPI**. El administrador carga los productos en USD y la plataforma calcula en milisegundos los precios en Bolívares y Pesos Colombianos para el cliente.
- **WebSockets (Socket.io):** Alertas visuales y sonoras en tiempo real para nuevos pedidos y actualizaciones de inventario sin recargar la página.
- **Inventario Ultra Optimizado:** Carga diferida (Lazy Loading) y paginación por pestaña en el panel administrativo, permitiendo manejar decenas de miles de productos sin pérdida de rendimiento. Búsqueda *Fuzzy* del lado del servidor.
- **Gestión de Combos:** Creación de promociones uniendo productos individuales del almacén, restando stock inteligentemente.
- **Mapa GPS Interactivo:** Los administradores pueden fijar la ubicación exacta de su local en un mapa interactivo.

🛠️ Stack Tecnológico

Frontend:

- React.js 19 + TypeScript
- Vite (Bundler ultra rápido)
- TailwindCSS + shadcn/ui (Diseño moderno, Glassmorphism)
- Zustand / Context API (Estado Global)
- React-Leaflet (Mapas Interactivos)

Backend:

- Node.js + Express.js
- Prisma ORM (Interacción con DB)
- PostgreSQL (Base de Datos Relacional)
- Socket.io (Comunicación bidireccional)
- Cloudinary (Almacenamiento de Imágenes)
- JWT (Autenticación)

📦 Instalación y Ejecución Local

1. **Clonar el repositorio:**

```
git clone https://github.com/JesusQuilleli/delivery-app.git
cd delivery-app
```

2. **Configurar Variables de Entorno:**
   - Crea un archivo `.env` dentro de la carpeta `backend/`.
   - Copia la estructura (puedes basarte en un `.env.example` si existe) y coloca tus claves de PostgreSQL, Cloudinary, etc.
3. **Iniciar Backend:**

```
cd backend
npm install
npx prisma migrate dev
npm run dev
```

4. **Iniciar Frontend:**

```
cd frontend
npm install
npm run dev
```

📚 Documentación y estado del proyecto

- `PLAN_DE_TRABAJO.md` — documento vivo con tareas, pruebas y bitácora. **Retomar por aquí después de cada sesión.**
- `Guia_Migracion_VPS.md` — recursos mínimos, despliegue, onboarding por tienda, pruebas multi-usuario y esquema de negocio.
- `Documentacion_Completa.md` — arquitectura, funcionalidades y detalles técnicos.
- `Flujo_Completo_App.md` — recorrido funcional de cliente, administrador y motorizado.
- `Brochure_Comercial.md` — presentación para clientes.
- `Documentacion_TiendaFast.pdf` — PDF consolidado con toda la documentación, recursos y esquema de negocio.

📝 Documentación Comercial

Si deseas ver las funcionalidades desde una perspectiva de negocio (para presentaciones a clientes o ventas), revisa `Brochure_Comercial.md` o el PDF consolidado.